

浙江大学

本科生毕业论文(设计)

文献综述和开题报告



姓名与学号 钱周越 3220104074

指导教师 李雨文教授

年级与专业 2022 级 信息与计算科学

所在学院 数学科学学院

一、题目：基于增量奇异值分解的偏微分方程模型降阶方法

二、指导老师对文献综述、开题报告、外文翻译的具体要求

文献综述不少于 3000 字，中外文献查阅篇数不少于 8 篇，其中外文文献建议为 3-5 篇。开题报告不少于 3500 字。外文翻译译文不少于 3000 字，译文须标注文章标题和原作者姓名，并要求提供外文原文及其出处。

文献综述要求

- (1) 简述与有限元方法、POD 方法以及缩减基方法 (Reduced Basis Method) 相关的教材和论文内容，掌握当前该领域的研究基础，并分析毕业论文研究内容与这些文献之间的联系与差异。
- (2) 分析 POD 方法以及本文拟研究的增量 POD 方法，简要说明这些方法的优缺点，并结合已有文献提出个人思考。
- (3) 概述所查阅参考文献的主要创新点，并指出其中存在的问题或尚未解决的问题。
- (4) 从所阅读的外文文献中选择一篇进行完整翻译，要求结构完整、语句通顺，并提供外文原文及其出处。

开题报告要求

- (1) 简述模型降阶方法以及 POD 方法的应用背景。
- (2) 在文献综述基础上分析 POD 方法的缺陷，并说明研究增量 POD 方法的必要性。
- (3) 从理论分析与数值实验两个方面说明选题的可行性。
- (4) 阐述论文的主要研究内容。
- (5) 根据研究内容制定具体的实施计划与研究步骤。
- (6) 明确论文最终预期达到的研究成果。

指导教师（签名）_____

2026 年 3 月 6 日

目录

一、文献综述	1
1 研究背景	1
2 国内外研究进展	3
2.1 参数化 PDE 的 Reduced Basis 与 Offline/Online 框架	3
2.2 POD/SVD 及其在降阶中的作用	3
2.3 面向 PDE 数据的加权内积与 core SVD	4
2.4 Brand 增量 SVD 算法及其意义	5
2.5 Brand 2006 及后续改进：从直接更新到间接更新	8
3 存在问题	9
3.1 batch SVD 的重复分解与存储开销	9
3.2 Brand 增量 SVD 的正交性退化问题	9
3.3 Brand 开放问题与 Zhang (2022) 的回答	10
4 研究展望	11
5 参考文献	13
二、开题报告	14
1 问题提出的背景	14
1.1 背景介绍	15
1.2 POD 方法的局限性与增量 POD 的必要性	17
2 论文的主要内容和技術路线	18
2.1 主要研究内容	18
2.2 技术路线	19
2.3 可行性分析	20
3 研究计划进度安排及预期目标	21
3.1 进度安排	21
3.2 预期目标	22
4 参考文献	23
三、外文翻译	24

四、外文原文 50

《浙江大学本科生文献综述和开题报告考核表》

一、文献综述

1 研究背景

偏微分方程模型广泛出现于热传导、渗流、弹性力学、流体力学、电磁散射以及多物理场耦合问题中。在这些应用里，一个极其常见的任务并不是只求解一次方程，而是对同一类模型在不同参数下进行大量重复求解。例如，在参数辨识中需要不断试探候选参数；在最优控制与优化设计中需要反复评估目标函数；在不确定性量化中需要对参数空间进行大量采样；在实时预测与快速决策中又要求每次求解尽可能快。对于这类任务，单次求解的代价固然重要，但更关键的是总体重复求解成本，因此通常被称为 **many-query 场景**。

以参数化椭圆型方程为例，设

$$-\nabla \cdot (a(x; \mu) \nabla u(x; \mu)) = f(x), \quad x \in \Omega, \mu \in \mathcal{P} \subset \mathbb{R}^p.$$

经过有限元离散后，可得到大规模线性系统

$$A(\mu)u_h(\mu) = f(\mu), \quad A(\mu) \in \mathbb{R}^{N \times N}, u_h(\mu) \in \mathbb{R}^N, N \gg 1.$$

当参数 μ 改变时，需要反复求解不同的高维线性系统。如果直接对每一个参数点都做一次高精度有限元求解，那么在 **many-query 场景**下，离散维数 N 与参数样本数 m 的叠加会迅速推高总成本。因此，如何在保证精度的前提下显著降低多参数重复求解开销，成为参数化偏微分方程数值计算中的核心问题。

正是在这一背景下，**Reduced Basis (RB) 方法**成为参数化 PDE 的核心降阶技术之一。RB 的基本思想是：大量高维解虽然存在于 $\mathbb{R}^N$ 中，但对于许多参数化问题，它们往往集中在一个低维流形附近，因此可以用一个维数远小于 N 的低维子空间来逼近^[1-2]。若记降阶基矩阵为

$$V_r \in \mathbb{R}^{N \times r}, \quad r \ll N,$$

则可用

$$u_h(\mu) \approx V_r a(\mu)$$

来近似高维解，再通过 Galerkin 投影得到降阶系统

$$V_r^T A(\mu) V_r a(\mu) = V_r^T f(\mu).$$

这样一来，昂贵的高维求解被转化为低维系统求解。RB 方法最重要的思想之一，就是通过 Offline/Online 分解将主要计算负担集中到离线阶段，而把快速响应留给在线阶段^[1-2]。

那么，为什么 RB 往往又要与 SVD 结合？原因在于：RB 的关键并不只是“有一个低维空间”，而是要“构造一个尽可能优的低维空间”。最经典的做法是先收集若干高精度快照

$$U = [u_h(\mu_1), u_h(\mu_2), \dots, u_h(\mu_m)] \in \mathbb{R}^{N \times m},$$

再对快照矩阵做奇异值分解

$$U = \Phi \Sigma \Psi^T.$$

取前 r 个左奇异向量便得到 POD 基

$$V_r = \Phi(:, 1:r).$$

这一步实际上是在 Frobenius 范数意义下寻找对所有快照的最优低秩逼近，因此 POD/SVD 在 RB 中并不是可有可无的附属步骤，而是构造高质量降阶基的核心工具。

进一步的问题是：为什么还要引入 **增量 SVD**？这是因为在实际的 RB 离线阶段，快照往往并不是一开始就全部拿到的，而是随着参数采样逐步生成的。一方面，贪婪采样、自适应采样等策略都天然带有“迭代新增样本”的结构；另一方面，在大规模仿真中，一次性存储全部快照矩阵本身就可能带来显著的内存压力。因此，若每新加入一个快照都重新对全部快照矩阵做一次 batch SVD，那么计算与存储代价都会很高。

因此，从 many-query 的整体计算链条看，逻辑是清楚的：many-query 问题需要一个高效的参数化求解框架，因此选择 RB；RB 需要构造尽可能优的低维基，因此自然结合 POD/SVD；而 POD/SVD 在实际离线构基中又面临快照逐步

增长、重复分解代价高的问题，因此进一步需要增量 SVD^[3-5]。

2 国内外研究进展

2.1 参数化 PDE 的 Reduced Basis 与 Offline/Online 框架

参数化 PDE 的模型降阶研究在过去二十余年发展迅速，其中最具代表性的两条主线是 POD 与 RB。Patera、Rozza 以及 Hesthaven 等人的工作系统奠定了参数化方程降阶求解的理论与算法框架：一方面，解流形在适当条件下具有较低的 Kolmogorov 宽度，说明低维近似具有理论可行性；另一方面，Offline/Online 分解将计算量大的部分集中在离线阶段，将快速求解保留到在线阶段，使 many-query 计算成为可能^[1-2]。

具体而言，RB 方法的优势在于它并不追求在整个高维空间内逼近，而是针对参数族解的实际分布构造一个问题相关的近似空间。对于许多椭圆型和抛物型问题，参数变化虽然会改变系数矩阵与解向量，但其主导模态常常只占据一个较低维的子空间，因此 RB 可以用很小的 r 达到较高精度。与此同时，若系统具有仿射参数分解，还可以进一步预组装在线阶段的矩阵和向量，从而将在线复杂度降到与 r 而非 N 相关^[1-2]。

不过，RB 的瓶颈也非常明确：离线构基。构基质量直接决定在线精度，而构基代价又往往主导整体成本。特别是在样本数较大、单次高精度求解昂贵、快照维数很高时，如何高效地从快照中提取主导信息就成为核心问题。这也是后续 POD/SVD 与增量 SVD 被引入 RB 框架的根本原因^[1,6]。

2.2 POD/SVD 及其在降阶中的作用

Proper Orthogonal Decomposition 本质上是一个最优低秩逼近方法。对于快照矩阵

$$U = [u_1, \dots, u_m] \in \mathbb{R}^{N \times m},$$

其截断 SVD

$$U \approx U_r = \Phi_r \Sigma_r \Psi_r^T$$

满足

$$\|U - U_r\|_F^2 = \sum_{i>r} \sigma_i^2,$$

即在所有秩不超过 r 的矩阵中，截断 SVD 给出了最小的 Frobenius 误差。这意味着，如果用前 r 个左奇异向量张成的空间去逼近快照集合，那么从总体均方误差意义看，它是最优的^[7]。

在 PDE 降阶中，POD 有两层意义。第一层是数据压缩：把海量高维快照压缩到少数主模态上。第二层是物理解释：奇异值衰减速度反映了解流形的有效维数，主导模态往往对应主要物理结构。对于 many-query 问题，如果快照在主模态方向上高度集中，那么说明用低维基来代替全维求解具有很强可行性^[1,7]。

然而，经典 POD/SVD 也有天然缺陷。它是典型的 batch 处理方式：需要在构造好全部快照矩阵之后一次性做分解。在离线阶段，这种模式对内存与算力都不友好。特别是在有限元背景下， N 很大，而参数采样数 m 也可能逐步增多，此时反复对大矩阵做完整 SVD 的代价十分显著。

2.3 面向 PDE 数据的加权内积与 core SVD

对于一般的数据矩阵，标准 Euclidean 内积下的 SVD 已足够。但在有限元离散的 PDE 数据里，更自然的内积通常并不是标准内积，而是由质量矩阵或刚度矩阵诱导的加权内积。例如在有限元空间中，若 M 为质量矩阵，则常考虑

$$\langle u, v \rangle_W = u^T W v, \quad W = M.$$

这意味着正交性、范数与能量贡献的定义都应当相对于 W 而不是单位矩阵。若忽略这一点，所构造的 POD 基就可能与连续问题中的自然能量结构不一致。

为了解决这一问题，Fareed、Singler、Zhang 与 Shen 等人发展了适用于 PDE 仿真数据的 **incremental POD / core SVD** 框架。在这一框架下，不再简单追求 $Q^T Q = I$ ，而是要求

$$Q^T W Q = I, \quad R^T R = I,$$

并将

$$U = Q \Sigma R^T$$

称为 **core SVD**。Zhang (2022) 也在文章中沿用了这一定义，并明确指出：若数据来自 Galerkin 型 PDE 仿真，则直接应用标准 Brand 算法会牵涉到 Cholesky 分解，而通过 core SVD 可以更自然地在 weighted inner product 下工作^[5-6]。

这一步对基于有限元快照的降阶问题尤其关键，因为研究对象不是一般图像流或机器学习数据，而是有限元离散下的参数化 PDE 快照，因此加权内积框架下的 core SVD 比标准 Euclidean SVD 更符合问题本身的数值结构。

2.4 Brand 增量 SVD 算法及其意义

增量奇异值分解的核心思想，是在已有矩阵奇异值分解结果的基础上，随着新列向量不断加入，快速更新当前分解，而不必每次都对扩展后的整体矩阵重新执行一次 batch SVD。对于快照逐步生成的 POD/RB 离线阶段，这种“边到达、边更新”的机制尤其重要，因此 Brand 的工作通常被视为增量 SVD 研究中的经典起点^[3-5]。

在有限元离散所诱导的加权内积框架下，设

$$\langle u, v \rangle_W = u^T W v,$$

其中 W 通常为质量矩阵或其他对称正定权矩阵。若矩阵 $U_\ell \in \mathbb{R}^{m \times \ell}$ 的前 ℓ 列已经得到了一个截断的 core SVD

$$U_\ell = Q \Sigma R^T, \quad Q^T W Q = I, \quad R^T R = I,$$

其中 $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{\ell \times k}$, 且 $k \leq \ell$, 那么 Brand 型算法的目标就是在加入新列 $u_{\ell+1}$ 后，快速构造

$$U_{\ell+1} = [U_\ell, u_{\ell+1}]$$

的更新分解^[3,5]。

这一算法可以分为“初始化—更新—重正交化”三个层次。

2.4.1 初始化

若只考虑首个非零列向量 u_1 ，则初始化非常直接。此时

$$\Sigma = (u_1^T W u_1)^{1/2}, \quad Q = u_1 \Sigma^{-1}, \quad R = 1.$$

于是便得到单列矩阵 U_1 的初始 core SVD。这个初始化步骤的本质，是先用第一个快照建立一个一维的加权正交基方向，并把对应的尺度信息吸收进奇异值 Σ

中^[5]。

2.4.2 更新

假设当前已有

$$U_\ell = Q\Sigma R^T.$$

当新快照 $u_{\ell+1}$ 到来时，首先将其投影到当前左奇异子空间 $\text{span}(Q)$ 上，并计算残差：

$$d = Q^T(Wu_{\ell+1}), \quad e = u_{\ell+1} - Qd, \quad p = (e^T W e)^{1/2}.$$

这里， d 表示新列在当前基上的坐标， e 表示新信息中无法被现有子空间表示的部分， p 则衡量该残差的加权范数。若 $p > 0$ ，进一步令

$$\tilde{e} = e/p.$$

这样便可写出如下恒等分解：

$$U_{\ell+1} = [Q, \tilde{e}] \begin{bmatrix} \Sigma & Q^T W u_{\ell+1} \\ 0 & p \end{bmatrix} \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^T.$$

记中间小矩阵为

$$Y = \begin{bmatrix} \Sigma & d \\ 0 & p \end{bmatrix},$$

则更新问题便被转化为求一个只有 $(k+1) \times (k+1)$ 规模的小矩阵 Y 的 SVD。设

$$Y = Q_Y \Sigma_Y R_Y^T,$$

则理论上就可以更新得到

$$Q^+ = [Q, \tilde{e}] Q_Y, \quad \Sigma^+ = \Sigma_Y, \quad R^+ = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y.$$

这正是 **Brand** 型增量更新的核心：把对大矩阵 $U_{\ell+1}$ 的重新分解，等价转化成对一个低维边界矩阵 Y 的分解，因此显著降低了更新代价^[3,5]。

2.4.3 截断策略

在实际应用中，增量 SVD 往往并不追求完整秩，而更关注主导模态，因此需要配合截断机制。Brand 型算法中至少存在两类自然截断：

第一类是**残差截断**。如果

$$p < \text{tol},$$

说明新快照 $u_{\ell+1}$ 与已有子空间非常接近，此时可近似认为 $p = 0$ 、 $\tilde{e} = 0$ ，即新增样本并未真正带来新的独立方向。于是只需更新已有模态，不必增加秩。此时常写成

$$\begin{aligned} Q &\leftarrow QQ_Y(1:k, 1:k), & \Sigma &\leftarrow \Sigma_Y(1:k, 1:k), \\ R &\leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(:, 1:k). \end{aligned}$$

第二类是**奇异值截断**。若更新后最后若干个奇异值小于给定阈值，则可将其丢弃，仅保留前 r 个主导模态，以控制秩增长并抑制机器精度附近的伪小奇异值。这一策略在面向 PDE 快照的实际 POD/RB 构基中尤其重要，因为目标往往是稳定提取低维主子空间，而不是恢复所有数值上微弱的方向^[5-6,8]。

2.4.4 整体算法结构

综合来看，Brand 型增量 SVD 的“完全增量版”可以概括为如下流程：先用首列初始化当前分解；随后对每一个新到达的列向量执行更新；每一步更新后，再根据需要进行重正交化修正。其结构可以形式化地写成：先取 u_1 建立初始的 (Q, Σ, R) ；然后对 $\ell = 2, \dots, n$ ，依次读入 u_ℓ ，执行更新，再执行重正交化，最终返回更新后的三元组 (Q, Σ, R) ^[3-5]。

从 RB/POD 离线构基的角度看，这一算法的重要意义在于：它使得快照矩阵可以按列流式到达，而低维基空间可以同步更新，从而避免“每来一个快照就重做一次 batch SVD”的高额代价。因此，在 many-query 场景下，它不仅是一个数值线性代数技巧，更是连接“高精度快照生成”和“低维缩减基提取”的关键中间环节。

2.5 Brand 2006 及后续改进：从直接更新到间接更新

尽管上述直接更新形式已经大幅优于反复执行整体 SVD，但如果每一步都显式构造

$$Q^+ = [Q, \tilde{e}]Q_Y, \quad R^+ = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y,$$

那么随着样本数不断增长，左右奇异向量矩阵仍会越来越大，更新成本也会逐步上升。Brand 在后续工作中进一步关注低秩修改问题，试图减少对外层大矩阵的频繁操作，把更多更新压缩到低维空间内部完成^[4]。

沿着这一思路，后续研究逐渐发展出一种更细致的分层结构：将奇异向量更新拆成“大外层矩阵 + 小正交因子”的乘积形式。这样，大矩阵主要负责记录当前子空间的外壳，而频繁发生的旋转与修正则尽可能留在小矩阵内部完成。Zhang (2022) 在此基础上进一步分析了这种结构在 weighted inner product 下的数值行为，并指出：从复杂度角度看，把更新尽量压缩到小矩阵层面是合理的；但从稳定性角度看，必须仔细区分“哪些正交因子容易失去正交性，哪些则相对稳定”，否则算法虽然快，却未必可靠^[5]。

因此，从 Brand 2002 到 Brand 2006，再到 Zhang 2022，可以看到增量 SVD 的研究主线已经不再只是“如何更新”，而逐渐转向“如何在高效更新的同时维持数值稳定与正交结构”。这也正是该方法从一般数据流处理走向 PDE 降阶构基时必须面对的问题。

2.5.1 增量 POD 的优势与思考

与传统 batch POD 相比，增量 POD 的首要优势在于其**更符合快照逐步生成的离线构基过程**。在参数化偏微分方程的 many-query 场景中，高精度快照往往不是一次性全部获得，而是随着参数采样、贪婪选点或自适应策略逐步产生。此时，若每增加一个快照都重新对全部快照矩阵执行一次 batch SVD，则不仅计算代价高，而且要求持续保存全部历史快照，带来较大的存储负担。增量 POD 则可以在已有分解基础上对新到达的数据进行递推更新，使低维基空间随样本增长同步演化，因此在算法结构上更适合实际的离线阶段。

第二，增量 POD 在**内存占用与可扩展性**方面具有明显优势。对于有限元离散得到的高维快照，状态维数 N 往往很大，而样本数 m 也可能随着参数探索不

断增加。传统 batch POD 通常需要保留完整快照矩阵 $U \in \mathbb{R}^{N \times m}$ ，当 N 和 m 同时较大时，存储压力会迅速增大。相比之下，增量 POD 只需维护当前的低秩分解结果及少量新增信息，而不必始终回看全部历史数据，因此更适合大规模、流式或长期累积的数据环境。

第三，增量 POD 更有利于与参数采样策略和在线构基过程耦合。在实际 reduced basis 方法中，快照的选取通常并非固定不变，而是与误差估计、贪婪算法、自适应加点等策略紧密相关。增量 POD 的递推更新机制使得“新增一个参数样本—更新一次缩减基”这一过程变得自然可行，从而更容易嵌入完整的 RB 离线流程。这说明增量 POD 的意义并不只是“把 SVD 算得更快”，而是在 many-query 框架下提供了一种更加灵活的基空间构造方式。

当然，增量 POD 的优势并不是无条件的。与 batch POD 相比，它最大的代价在于数值稳定性问题更加突出。由于更新过程是逐步递推的，舍入误差会在多次迭代后不断累积，进而导致正交性退化、奇异值扰动以及截断误差传播等问题；在有限元加权内积框架下，这一问题又会进一步加剧。因此，增量 POD 的真正关键不只是“增量更新”本身，而在于如何在更新效率与数值稳定性之间取得平衡。换言之，增量 POD 的研究重点应当从“能否递推更新”进一步转向“如何稳定地递推更新”。

3 存在问题

3.1 batch SVD 的重复分解与存储开销

首先，最直接的问题来自 batch SVD 本身。对于一个 $N \times m$ 的快照矩阵，一次完整分解就已不便宜；若在离线阶段每新增一个快照都重做一次分解，总成本会随样本数迅速增长。更重要的是，batch SVD 通常要求同时保存全部快照数据，这在高维有限元场景下意味着明显的内存压力。因此，对于快照逐步生成的 RB 离线阶段而言，batch SVD 在计算模式上并不理想^[3-4,6]。

3.2 Brand 增量 SVD 的正交性退化问题

相比 batch SVD，Brand 型增量 SVD 的最大优势在于每一步更新代价低；但它同时也带来了最核心的数值隐患，即正交性退化问题。理论上，如果在精确算术下每一步都满足

$$Q^T W Q = I, \quad R^T R = I,$$

则更新后的分解始终保持正交结构。然而在实际浮点运算中，矩阵乘法、归一化和重复迭代会不断引入舍入误差；当更新次数达到上千甚至上百万次时，这些误差会逐渐积累，使本应正交的向量组偏离正交条件^[4-5]。

这一问题之所以严重，是因为它会直接破坏增量 SVD 的数学含义。首先，一旦

$$Q^T W Q \neq I,$$

当前分解就不再是严格意义上的 **core SVD**，相应奇异值也不再准确反映真实谱结构。这样一来，以奇异值大小为依据的截断策略可能失效。其次，**POD/RB** 的核心是利用主导模态张成一个高质量低维子空间；若基向量之间已经失去正交性，则不同模态可能发生污染与冗余，从而降低缩减基质量，进一步影响在线阶段的精度与稳定性。最后，在有限元加权内积下，这一问题往往比标准 **Euclidean** 情形更棘手，因为重正交化必须针对 W -内积执行，代价显著增加^[5-6]。

Brand 型算法的完整流程中通常都会显式加入一个**重正交化步骤**。以 **weighted Gram-Schmidt** 为例，其基本思想是：当检测到正交性误差超过阈值时，依次对当前基向量做加权正交投影与归一化修正，即对第 i 个向量减去其在前面各向量方向上的 W -投影，再按 W -范数归一化。这个过程理论上可以恢复当前基的加权正交性，但在实践中代价很高，尤其当基维数和更新次数都较大时，重正交步骤可能占据算法的大部分耗时^[5]。

Zhang (2022) 的数值结果清楚地说明了这一点：在标准内积情形 $W = I$ 下，算法的整体代价尚可接受；但在有限元质量矩阵情形 $W = M$ 下，重正交化几乎成为主要耗时来源。文中甚至专门用

$$E_W = \|I - Q^T W Q\|$$

来衡量加权正交误差，并展示了在长时间迭代后，即使实施了重正交，正交性仍可能持续退化^[5]。这说明问题并不只是“要不要重正交”，而是 **Brand** 型更新结构本身就会持续制造需要修补的正交性误差。

3.3 Brand 开放问题与 Zhang (2022) 的回答

Brand 在 2006 年提出了一个非常关键的开放问题：为了保证总体数值精度，到底需要多频繁地执行重正交化？这一问题的难点不在于“理论上可不可以重正

交”，而在于：如果一味地频繁重正交，算法效率优势会被大幅削弱；但如果重正交不及时，奇异向量的正交结构又会逐渐崩塌^[4]。

Zhang (2022) 对这一开放问题的回答，并不是简单给出一个固定的“每多少步重正交一次”的经验法则，而是更深入地分析了 Brand 型分层更新结构中不同部分的数值脆弱性。其核心观点是：并不是所有正交因子都会同等程度地失去正交性。小规模内部因子由于维度低、连乘次数有限，往往可以在合理精度下保持稳定；真正容易快速退化的，反而是不断扩展的外层大矩阵。因此，算法设计的重点不应放在对所有小矩阵做无差别重正交，而应集中处理最外层、最脆弱的大尺度正交结构^[5]。

更具体地说，Zhang 提出应尽量避免反复形成并连乘大量小正交矩阵，而把必要的正交修正集中到新增方向相对于当前大基矩阵的递归加权正交化上。这样做的结果是：一方面保留了增量更新“小矩阵计算为主”的效率优势；另一方面又减少了由长链式正交矩阵乘法带来的正交性侵蚀。这实际上把 Brand 原先的开放问题从“多久重正交一次”转化为“应当对哪一层进行重正交”，从而给出了更具结构性的答案^[5]。

对于基于有限元快照的 PDE 模型降阶而言，这一点尤其重要。因为在加权内积框架下，重正交不仅昂贵，而且直接关系到 POD/RB 基的物理一致性与数值稳定性。因此，如何把 Brand 型增量更新与更稳健的重正交策略结合起来，正是当前这类课题最自然、也最有价值的研究切入点之一。

4 研究展望

结合上述研究进展与存在问题，可以看到“基于增量奇异值分解的偏微分方程模型降阶方法”具有清晰而具体的研究空间。未来可从以下几个方向深入展开。

第一，在参数化椭圆 PDE 上建立完整的算法验证链条。这一路径最符合本科毕业论文的工作量与可操作性。可选取典型变系数 Poisson 方程或扩散方程作为模型，先实现高精度 FEM 快照生成，再分别实现 batch SVD 构造的 POD/RB 作为基线，以及 weighted inner product 下的增量 core SVD 作为对照。随后系统比较三类指标：近似误差、正交性指标、计算时间与内存占用^[1,5-6]。

第二，围绕 weighted inner product 下的数值稳定性展开更细致研究。有限

元背景中的自然内积不是标准内积，因此单纯移植一般数据流领域的增量 SVD 算法并不充分。一个值得重点考察的问题是：不同 W -正交化策略对误差、耗时和鲁棒性的影响如何。例如，可比较完全重正交、按阈值触发重正交、仅对新增方向递归正交化等不同策略，并用

$$E_W = \|I - Q^T W Q\|$$

作为统一指标衡量正交性保持效果。^[5,8]。

第三，**研究截断策略与基维数控制**。增量 SVD 的价值并不只在“更新快”，还在于能边生成、边截断、边控制秩增长。可重点分析两类阈值：一类是残差阈值 $p < \text{tol}$ ，用来判断新快照是否基本落在当前子空间中；另一类是最小奇异值阈值 $\sigma_k < \text{tol}$ ，用来决定是否丢弃弱模态。不同阈值会同时影响误差、基维数、在线效率和正交性，因此值得系统实验比较^[3,5-6]。

第四，**从算法层面探索比 Brand 原始更新更稳健的结构**。这部分未必要提出全新的理论算法，但至少可以在实现和比较层面回答一个非常具体的问题：Brand 直接更新、Brand 间接更新、Zhang 型改进更新，在 PDE 快照数据上谁更适合做 POD/RB 离线构基。由于这一方向同时连接了计算效率、正交性保持与加权内积下的数值稳定性，因此即便论文最终侧重数值比较，也具有明确的方法学意义^[4-5]。

第五，**将研究结果进一步连接到更复杂的 many-query 任务**。如果时间允许，可以在论文结尾讨论其向更复杂模型的推广，包括抛物型方程、时变 PDE、非线性降阶，以及更复杂的采样机制如贪婪算法和自适应参数选点。这样做的意义在于说明：增量 SVD 并不只是一个线性代数技巧，而是 many-query 降阶计算链条中的关键中间环节，它直接影响离线构基效率，并最终影响在线快速求解的可行性^[1-2,5]。

综上，基于增量奇异值分解的 PDE 模型降阶方法并不是简单地把 SVD 换成增量 SVD，而是在 many-query、RB、POD、weighted inner product、数值正交性与截断控制之间建立起一套完整的方法链。已有研究表明，增量 SVD 在离线构基效率方面具有明显潜力，但其数值稳定性，尤其是加权内积下的正交性保持问题，仍然是进一步研究的关键所在^[3-6]。

5 参考文献

- [1] HESTHAVEN J S, ROZZA G, STAMM B. Springerbriefs in mathematics: Certified reduced basis methods for parametrized partial differential equations[M]. Springer, 2015.
- [2] PATERA A T, ROZZA G. Reduced basis approximation and a posteriori error estimation for parametrized partial differential equations[M]. MIT Pappalardo Graduate Monographs in Mechanical Engineering, 2007.
- [3] BRAND M. Incremental singular value decomposition of uncertain data with missing values [C]//European Conference on Computer Vision (ECCV). Springer, 2002: 707-720.
- [4] BRAND M. Fast low-rank modifications of the thin singular value decomposition[J/OL]. Linear Algebra and its Applications, 2006, 415(1): 20-30. DOI: [10.1016/j.laa.2005.07.021](https://doi.org/10.1016/j.laa.2005.07.021).
- [5] ZHANG Y. An answer to an open question in the incremental SVD[A]. 2022.
- [6] FAREED H, SINGLER J R, ZHANG Y, et al. Incremental proper orthogonal decomposition for PDE simulation data[J/OL]. Computers & Mathematics with Applications, 2018, 75(6): 1942-1960. DOI: [10.1016/j.camwa.2017.09.012](https://doi.org/10.1016/j.camwa.2017.09.012).
- [7] GOLUB G H, LOAN C F V. Matrix computations[M]. 4th ed. Johns Hopkins University Press, 2013.
- [8] FAREED H, SINGLER J R. Error analysis of an incremental proper orthogonal decomposition algorithm for PDE simulation data[J/OL]. Journal of Computational and Applied Mathematics, 2020, 368: 112525. DOI: [10.1016/j.cam.2019.112525](https://doi.org/10.1016/j.cam.2019.112525).

二、开题报告

1 问题提出的背景

参数化偏微分方程广泛出现于热传导、扩散、渗流、弹性力学、电磁场以及多物理场耦合等问题中。在许多实际应用里，研究者往往并不满足于对某一个固定参数下的模型求解一次，而是需要针对不同参数反复进行数值求解。例如，在参数识别问题中，需要不断调整候选参数并比较模型输出；在优化设计与最优控制中，需要对多个设计变量下的状态方程进行重复评估；在不确定性量化中，需要对参数空间进行大量采样；在实时预测、快速决策和数字孪生等场景中，则要求在参数变化后尽快给出新的近似解。对于这类问题，关键困难不在于单次求解，而在于总体重复求解成本过高，因此常称为 **many-query 场景**。

以参数化椭圆型方程为例，考虑

$$-\nabla \cdot (a(x; \mu) \nabla u(x; \mu)) = f(x), \quad x \in \Omega, \mu \in \mathcal{P} \subset \mathbb{R}^p,$$

配以适当边界条件。经过有限元离散后，可得到大规模代数系统

$$A(\mu)u_h(\mu) = f(\mu), \quad A(\mu) \in \mathbb{R}^{N \times N}, u_h(\mu) \in \mathbb{R}^N,$$

其中 N 为离散自由度数，通常远大于 1。当参数 μ 在参数域 $\mathcal{P}$ 中变化时，如果对每一个参数点都独立进行一次高精度有限元求解，那么随着参数样本数的增加，总计算成本将迅速增大。因此，如何在保证近似精度的前提下显著降低重复求解开销，是参数化 PDE 数值计算中的重要问题。

为解决这一问题，模型降阶方法（Model Order Reduction, MOR）得到了广泛研究。其中，**Reduced Basis (RB) 方法**是参数化 PDE 中最具代表性的降阶框架之一^[1-3]。其基本思想是：虽然每个高精度解都位于高维空间 $\mathbb{R}^N$ 中，但对于许多参数化问题，不同参数对应的解往往集中在某个低维流形附近，因此可以构造一个维数远小于 N 的低维子空间来逼近高维解。

若记降阶基矩阵为

$$V_r \in \mathbb{R}^{N \times r}, \quad r \ll N,$$

则可以近似表示

$$u_h(\mu) \approx V_r a(\mu),$$

再通过 Galerkin 投影得到低维系统

$$V_r^T A(\mu) V_r a(\mu) = V_r^T f(\mu).$$

然而, RB 方法的关键并不只是在于“降维”,更在于如何构造一个尽可能优的低维基空间。最常见的方法是收集若干高精度快照解

$$U = [u_h(\mu_1), u_h(\mu_2), \dots, u_h(\mu_m)] \in \mathbb{R}^{N \times m},$$

并通过奇异值分解

$$U = \Phi \Sigma \Psi^T$$

提取前若干个最重要的左奇异向量,作为 POD/RB 基。这种方法在 Frobenius 范数意义下给出最优低秩逼近,因此在模型降阶中具有核心地位^[4-5]。

进一步的问题是,在 many-query 场景下,快照往往是随着采样过程逐步生成的,而不是一开始就全部获得。如果每增加一个快照都重新对全部快照矩阵做一次 batch SVD,则不仅计算代价高,而且需要较大的存储开销。

基于这一背景,增量奇异值分解 (incremental SVD) 成为一个自然且重要的研究方向。Brand 在文献^[6-7]中提出了经典的增量 SVD 更新方法,可以在新数据到达时更新奇异值分解,而无需重新分解整个矩阵。

1.1 背景介绍

从数值分析角度看,参数化 PDE 的 many-query 计算具有典型的“离线贵、在线快”的需求结构。RB 方法通常采用 Offline/Online 分解:离线阶段生成快照、构造基空间;在线阶段仅在低维空间内求解,从而大幅减少计算量^[1]。

设快照矩阵为

$$U = [u_1, u_2, \dots, u_m],$$

经典方法需要对其统一执行 SVD 分解。而在实际构基中,快照往往逐步生成,因此矩阵具有按列扩展的结构。

在 **weighted inner product** 下，若

$$U_\ell = Q\Sigma R^T, \quad Q^T W Q = I,$$

则称其为 **core SVD**。**Brand** 型增量算法通过将新增快照分解为投影部分和残差部分：

$$d = Q^T(Wu_{\ell+1}), \quad e = u_{\ell+1} - Qd, \quad p = (e^T W e)^{1/2},$$

从而构造更新矩阵

$$Y = \begin{bmatrix} \Sigma & d \\ 0 & p \end{bmatrix}.$$

对该小矩阵执行 **SVD**，即可完成整体更新。

从数学角度看，**POD** 方法与奇异值分解之间存在紧密联系。设快照矩阵为

$$U = [u_1, u_2, \dots, u_m] \in \mathbb{R}^{N \times m},$$

其奇异值分解为

$$U = \Phi \Sigma \Psi^T,$$

其中 Φ 和 Ψ 分别为左右奇异向量矩阵， Σ 为奇异值矩阵。经典结果表明，取前 r 个左奇异向量所张成的子空间在 **Frobenius** 范数意义下给出了对快照矩阵的最优秩- r 逼近^[5]。因此，**POD** 基可以被理解为快照数据能量最大的主导模态。

然而，在参数化 **PDE** 的实际计算中，快照数量往往不断增加，如果每一次新增快照都重新执行完整 **SVD**，则离线阶段计算成本将迅速增长。因此，如何在已有 **SVD** 基础上进行高效更新，成为模型降阶中的重要问题。增量奇异值分解正是在这一背景下提出，其目标是在新数据到达时更新奇异值分解，而不必重新处理全部数据。

近年来，增量 **POD** 方法也被用于 **PDE** 仿真数据处理中^[8-9]。此外，**Zhang** 在^[10] 中研究了 **weighted inner product** 下的增量 **SVD** 算法，并分析了其数值稳定性。

1.2 POD 方法的局限性与增量 POD 的必要性

POD 方法由于能够从快照数据中提取主导模态，并在 Frobenius 范数意义下给出最优低秩逼近，因此已成为模型降阶中最常用的构基方法之一。然而，经典 POD 方法通常建立在 **batch SVD** 的基础上，即必须在收集全部快照之后，对完整快照矩阵一次性执行奇异值分解。这样的处理方式在理论上是清晰的，但在参数化 PDE 的实际计算中存在若干局限性。

首先，**经典 POD 的计算代价较高**。设快照矩阵为

$$U = [u_1, u_2, \dots, u_m] \in \mathbb{R}^{N \times m},$$

其中 N 是有限元离散后的空间维数， m 是快照数量。在参数化 PDE 的 many-query 场景下， N 往往很大，而 m 也会随着参数采样不断增加。此时，若始终采用 batch SVD，则需要反复对大规模矩阵进行分解，离线阶段的计算成本会迅速升高。尤其当每增加一个新快照都重新对整个快照矩阵做一次 SVD 时，这种重复计算会显著削弱模型降阶在整体流程中的效率优势。

其次，**经典 POD 对快照存储的要求较高**。batch SVD 通常要求在离线阶段保存全部历史快照，以便统一构造快照矩阵并进行分解。然而，在有限元背景下，每个快照本身就是一个高维向量，当快照数量逐步增加时，存储完整矩阵会带来明显的内存压力。对于大规模参数扫描、长时间仿真或流式生成数据的场景，这一问题尤为突出。

再次，**经典 POD 不适合快照逐步到达的构基过程**。在实际 reduced basis 方法中，快照往往不是预先全部给定的，而是随着贪婪选点、自适应采样或参数扫描过程逐步生成的。换言之，快照矩阵在离线阶段常常具有“按列不断扩展”的结构。如果仍坚持每次新增样本后都重新执行完整 SVD，则方法在算法结构上与实际数据生成机制并不匹配。因此，经典 POD 更适合静态、一次性的数据处理，而不够适合逐步更新的离线构基过程。

基于以上原因，研究**增量 POD**方法具有明显必要性。所谓增量 POD，是指在新快照到达时，不必重新处理全部历史数据，而是在已有分解结果的基础上递推更新当前低维基空间。本文中所讨论的增量 POD，主要通过对快照矩阵进行**增量奇异值分解 (incremental SVD)** 来实现。其核心思想是：当新的快照列向

量加入时，将其分解为相对于当前基空间的投影部分与正交残差部分，再通过一个小规模矩阵的奇异值分解完成整体更新。这样就避免了对整个扩展后快照矩阵重复做 batch SVD。

与经典 POD 相比，增量 POD 主要具有以下几个方面的优势。第一，它能够**适应快照逐步生成的离线流程**，与参数化 PDE 中常见的贪婪采样、自适应选点等策略更加匹配。第二，它能够**减少重复分解带来的计算负担**，从而提高离线构基效率。第三，它能够**降低对完整历史快照存储的依赖**，因而在大规模问题中更具可扩展性。第四，它使得“每增加一个样本就同步更新降阶基”成为可能，从而为更灵活的构基方式提供了算法基础。

当然，增量 POD 并不是对经典 POD 的简单替代。与 batch POD 相比，它虽然在效率和存储方面更具优势，但也带来了新的数值问题。最典型的问题是：在多次递推更新过程中，浮点误差会逐渐积累，导致基向量的正交性退化，进而影响奇异值的准确性、截断策略的可靠性以及最终降阶基的质量。特别是在有限元离散诱导的 weighted inner product 框架下，这一问题会更加明显。因此，增量 POD 的研究重点不仅在于“如何更快地更新”，也在于“如何在更新过程中保持数值稳定性”。

2 论文的主要内容和路线

2.1 主要研究内容

本文拟以参数化椭圆型偏微分方程为研究对象，围绕“增量奇异值分解在 PDE 模型降阶中的应用”展开研究，主要内容包括以下几个方面。

第一，建立参数化 PDE 模型降阶的基本框架。本文将从参数化椭圆型方程出发，给出其有限元离散形式，并说明在 many-query 场景下直接求解高维系统的成本问题。在此基础上，引入 Reduced Basis 方法和 POD 构基思想，说明快照矩阵的低秩结构为何能够支持模型降阶，以及 SVD 在构造近似最优低维基中的作用。

第二，分析 weighted inner product 下的 core SVD 结构。由于有限元离散通常伴随质量矩阵，因此本文将重点讨论加权内积

$$\langle u, v \rangle_W = u^T W v$$

下的正交性与矩阵分解形式，解释为何在 PDE 快照处理中应当考虑 **core SVD**，而不是简单套用标准 **Euclidean SVD**。这里的目标不是建立全新的抽象理论，而是在已有定义基础上，准确梳理其与有限元离散和 **POD** 构基之间的关系。

第三，研究增量 **SVD** 的初始化与更新机制。设快照矩阵按列逐步生成，本文将分析在已知

$$U_\ell = Q\Sigma R^T$$

的条件下，如何通过新增列向量 $u_{\ell+1}$ 构造

$$U_{\ell+1} = [U_\ell, u_{\ell+1}]$$

的更新分解。重点包括：投影项与残差项的分离、小规模边界矩阵 Y 的构造、更新后左右奇异向量与奇异值的表达方式，以及当残差很小或新奇异值过小时的截断处理。

第四，讨论重正交与数值稳定性问题。增量 **SVD** 的主要数值风险在于正交性退化，因此本文将分析为什么浮点误差会导致

$$Q^T W Q \neq I,$$

以及这种退化对奇异值真实性、截断可靠性和降阶基质量的影响。在此基础上，讨论加权 **Gram-Schmidt** 型重正交化的作用及其计算代价，并尝试比较不同策略下的数值表现。

第五，设计数值实验并进行结果比较。本文计划选取一个或若干典型的参数化椭圆型方程作为实验对象，生成有限元快照数据，并分别采用 **batch SVD** 与增量 **core SVD** 构造 **POD/RB** 基。比较指标包括：奇异值衰减、降阶逼近误差、加权正交性误差、离线时间开销以及内存使用情况。通过这些结果，评估增量 **SVD** 在 PDE 模型降阶中的适用范围与优势。

2.2 技术路线

本文拟采用“理论梳理—算法实现—数值比较”的技术路线。

第一步，完成文献阅读与问题建模。首先系统阅读参数化 PDE 模型降阶、**RB/POD** 方法、增量 **SVD** 以及 **weighted inner product** 下 **core SVD** 相关文献，明

确课题中的数学对象、算法目标与关键困难。随后选定一个典型的参数化椭圆型方程作为主要测试模型，并确定其边界条件、参数形式和有限元离散方案。

第二步，建立高精度快照生成模块。对选定的参数化 PDE 进行有限元离散，得到高维代数系统

$$A(\mu)u_h(\mu) = f(\mu).$$

在给定参数采样集 $\{\mu_1, \dots, \mu_m\}$ 下，计算对应的高精度数值解，并将其组织为快照矩阵

$$U = [u_h(\mu_1), \dots, u_h(\mu_m)].$$

同时构造相应的权矩阵 W ，通常取有限元质量矩阵，以便后续进行 **weighted core SVD**。

第三步，实现 **batch SVD** 与增量 **SVD** 两套离线构基流程。对于 **batch** 方法，直接对快照矩阵做标准或加权意义下的分解并截断，得到 **POD/RB** 基；对于增量方法，则按照“初始化—更新—必要时重正交—截断”的流程，逐列处理快照矩阵。这里的重点是保证实现与数学公式一致，并对关键中间量如残差范数 p 、奇异值阈值和正交性误差进行记录。

第四步，构造降阶模型并进行在线测试。将所得到的低维基代入参数化椭圆方程的离散系统，建立降阶模型

$$V_r^T A(\mu) V_r a(\mu) = V_r^T f(\mu),$$

对若干测试参数进行在线求解，并将降阶解与高精度有限元解进行比较，从而评估不同构基方式对在线精度的影响。

第五步，分析实验结果并总结结论。比较 **batch SVD** 与增量 **SVD** 在离线时间、内存开销、正交性保持、降阶误差和基维数控制等方面的差异，分析不同截断阈值与重正交策略的影响，并据此总结增量 **SVD** 在参数化 PDE 模型降阶中的优缺点与适用条件。

2.3 可行性分析

从研究对象看，本文拟研究的参数化椭圆型方程属于数值分析与科学计算中较为成熟的一类模型，其有限元离散、矩阵组装与数值求解流程较为清晰，适

合作为本科阶段模型降阶研究的基础载体。因此，在问题设置上具有较强可行性。

从理论基础看，课题涉及的核心概念包括有限元方法、POD/RB 模型降阶、奇异值分解与 **weighted inner product** 下的 **core SVD**。这些内容虽然跨越 PDE 数值、矩阵计算和模型降阶三个方向，但彼此联系紧密，且已有较为清楚的文献基础。本文的目标不是从零建立全新的深层理论，而是在已有理论框架下完成准确梳理、合理实现与数值分析，因此对于本科毕业论文而言是合适的。

从技术实现看，本课题所需的主要计算步骤包括：参数化 PDE 的有限元求解、快照矩阵构造、SVD 或增量 SVD 实现、重正交处理以及降阶误差比较。这些步骤均可借助现有数值计算环境完成，例如 MATLAB、Python 或其他常用科学计算工具。特别是增量 SVD 的更新公式本身由小矩阵 SVD 和向量投影构成，实现难度可控，便于逐步调试和验证。

从工作量看，若将研究重点聚焦在“参数化椭圆方程 + **weighted incremental core SVD** + 数值比较”这一主线上，则论文内容既有足够的数学深度，又不至于因为问题过大而难以完成。相比试图证明过于复杂的理论结果，这种“数学框架清晰、算法流程完整、实验结果可展示”的选题更符合本科毕业论文的时间与能力边界。

此外，从算法复杂度角度看，增量 SVD 方法在处理大规模快照矩阵时具有明显优势。传统 **batch SVD** 的计算复杂度通常为 $\mathcal{O}(Nm^2)$ 或 $\mathcal{O}(N^2m)$ ，当快照维数 N 和样本数量 m 都较大时，计算成本十分昂贵。相比之下，增量 SVD 每次只需要对一个 $(r+1) \times (r+1)$ 的小矩阵执行 SVD，其计算复杂度显著降低。因此，在 **many-query** 场景下，采用增量 SVD 构造降阶基不仅可以减少计算时间，还可以降低内存存储需求，从而提高离线阶段的整体效率。

3 研究计划进度安排及预期目标

3.1 进度安排

第一阶段：文献调研与问题细化。系统阅读参数化 PDE、Reduced Basis 方法、POD/SVD、incremental SVD 以及 **weighted core SVD** 相关文献，梳理课题中的基本概念、主要算法与已有结果。在此基础上，明确本文所采用的参数化椭圆方程模型、有限元离散方式以及数值实验平台。

第二阶段：建立高精度模型与快照生成程序。完成参数化椭圆型方程的有限元离散与求解程序，测试不同参数下高精度数值解的正确性与稳定性。随后生成训练快照集与测试快照集，为后续 batch SVD 和增量 SVD 构基提供数据基础。

第三阶段：实现 batch SVD 与增量 core SVD 算法。首先实现标准的快照矩阵分解与 POD 基提取过程，作为比较基线；再实现 weighted inner product 下的增量更新、截断与重正交步骤，并逐步验证更新公式是否正确、正交性是否保持以及截断策略是否有效。

第四阶段：建立降阶模型并开展数值实验。基于两种构基方式得到的低维基，建立参数化椭圆方程的降阶模型，比较其在不同参数点上的近似误差、离线耗时、内存消耗与正交性误差。对实验结果进行整理，并尝试分析不同阈值、不同基维数下的表现差异。

第五阶段：整理结论并撰写论文。对已有理论梳理、算法流程和实验结果进行系统总结，完成文献综述、开题报告、正文初稿与修改稿的撰写，并根据指导教师意见进一步修订，最终形成完整毕业论文。

3.2 预期目标

第一，较系统地梳理参数化椭圆型 PDE 在 many-query 场景下面临的计算问题，说明 Reduced Basis 方法、POD/SVD 以及增量 SVD 之间的逻辑关系，形成较为清晰的理论背景与问题表述。

第二，在 weighted inner product 框架下，较准确地给出增量 core SVD 的数学描述，包括初始化、更新、截断和重正交等步骤，并解释其在有限元快照数据中的意义。

第三，完成一个可运行的数值实验流程：从高精度有限元快照生成，到 batch SVD 与增量 SVD 构基，再到降阶模型测试与结果比较。通过实验展示增量 SVD 在离线构基中的潜在优势，以及其数值稳定性方面需要关注的问题。

第四，形成一篇结构完整、数学表述准确、实验结果明确的本科毕业论文。论文不追求过度扩张结论，而是力求在选定模型和算法框架下，给出可信、清楚、可复现的分析与比较结果，为后续进一步研究增量 SVD 在更复杂 PDE 模型中的应用打下基础。

4 参考文献

- [1] HESTHAVEN J S, ROZZA G, STAMM B. Certified reduced basis methods for parametrized partial differential equations[M]. Cham: Springer, 2015.
- [2] QUARTERONI A, MANZONI A, NEGRI F. Reduced basis methods for partial differential equations: An introduction[M]. Cham: Springer, 2016.
- [3] PATERA A T, ROZZA G. Reduced basis approximation and a posteriori error estimation for parametrized partial differential equations[M]. MIT Pappalardo Graduate Monographs, 2007.
- [4] KUNISCH K, VOLKWEIN S. Galerkin proper orthogonal decomposition methods for a general equation in fluid dynamics[J]. SIAM Journal on Numerical Analysis, 2002, 40(2): 492-515.
- [5] TREFETHEN L N, BAU D. Numerical linear algebra[M]. SIAM, 1997.
- [6] BRAND M. Incremental singular value decomposition of uncertain data with missing values [J]. European Conference on Computer Vision, 2002: 707-720.
- [7] BRAND M. Fast low-rank modifications of the thin singular value decomposition[J]. Linear Algebra and its Applications, 2006, 415(1): 20-30.
- [8] FAREED H, SINGLER J R, ZHANG Y. Incremental proper orthogonal decomposition for pde simulation data[J]. Computer Methods in Applied Mechanics and Engineering, 2018, 331: 92-121.
- [9] FAREED H, SINGLER J R. Online updating of proper orthogonal decomposition bases for time-dependent pdes[J]. SIAM Journal on Scientific Computing, 2020, 42(4): A2304-A2328.
- [10] ZHANG Y. An improved incremental singular value decomposition algorithm under weighted inner products[J]. Numerical Linear Algebra with Applications, 2022, 29(6): e2448.

三、外文翻译

增量 SVD 中一个开放问题的解答

张杨文

2022 年 5 月 3 日

摘要

增量奇异值分解由 Brand 提出, 用于高效计算矩阵的 SVD。该算法需要计算数千或数百万个正交矩阵并将它们相乘。然而, 乘法可能会破坏正交性。因此, 在实践中需要许多重正交化。在 [Linear Algebra and its Applications 415 (2006) 20-30] 中, Brand 问道: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度。” 在本文中, 我们回答了这个问题, 答案是我们可以避免计算大量这些正交矩阵, 因此通过修改他的算法, 重正交化并不是必需的。我们证明了这种修改不会改变算法的结果。数值实验说明了我们修改的性能。

1 引言

矩阵的奇异值分解在许多应用中都有应用, 例如本征正交分解模型降阶和主成分分析。

SVD 的一个缺点是计算成本高。设 U 是一个 $m \times n$ 的低秩 r 稠密矩阵, 传统方法的计算复杂度为 $O(mn^2 + m^2n + n^3)$ 时间, 这对于大规模矩阵是不可行的。Lanczos 方法 [2] 产生薄 SVD, 复杂度为 $O(mnr^2)$ 时间, 但我们需要提前知道秩。这些方法被称为批处理方法, 因为它们需要存储矩阵。

在某些场景中, 数据集是增量产生的, 例如时间相关偏微分方程的快照。随着矩阵的列变得可用时执行 SVD 可能是有利的, 而不是等待所有数据集可用后才开始计算。数据集的这些可用性特性催生了一类增量方法。

2002 年, Brand [3] 提出了一种新算法来增量地计算矩阵的 SVD。给定矩阵 $U = Q\Sigma R^T$ 的 SVD, 目标是通过使用 U 的 SVD 和新添加的向量 c 来更新相关矩阵 $[U \mid c]$ 的 SVD。然后可以通过以下步骤构造 $[U \mid c]$ 的 SVD:

1. 令 $e = c - QQ^\top c$ 且 $p = \|e\|$
2. 找到 $\begin{bmatrix} \Sigma & Q^\top c \\ 0 & p \end{bmatrix} = \tilde{Q}\tilde{\Sigma}\tilde{R}^\top$ 的完全 SVD, 然后
3. 通过下式更新 $[U \mid c]$ 的 SVD

$$[U \mid c] = ([Q \mid e/p]\tilde{Q})\tilde{\Sigma}\left(\begin{bmatrix} R & 0 & 0 \end{bmatrix}\tilde{R}\right)^\top.$$

对于许多激励性应用, 只需要 U 的主要奇异向量和值。因此, 在实践中, 当 p 或 $\tilde{\Sigma}$ 的最后一个奇异值很小时, 我们执行截断。增量 SVD 已应用于许多不同领域; 更多细节见 [8, 10-12]。还有许多其他增量方法, 见 [1] 的综述。

有两种更新左右奇异子空间的方法。第一种方法非常直接, 更新由下式给出

$$Q \leftarrow [Q \mid e/p]\tilde{Q}, \quad \Sigma \leftarrow \tilde{\Sigma}, \quad R \leftarrow \begin{bmatrix} R & 0 & 0 \end{bmatrix}\tilde{R}. \quad (1.1)$$

然而, 这种更新有两个缺陷:

- (1) (1.1) 的计算成本很高, 因为更新涉及非常高而瘦的矩阵 Q 和 R 。
- (2) 几乎每次更新后都需要对 Q 进行重正交化。理论上, 矩阵 $[Q \mid e/p]\tilde{Q}$ 是正交的。然而, 在实践中, 微小的数值误差会导致正交性损失。如果没有重正交化, 增量 SVD 算法的分解就不是正交的。因此, 奇异值不真实, 截断不可靠。因此需要重正交化, 如果矩阵的秩不是非常低, 计算成本可能会很高。具体来说, Fareed 等人在 [7] 中推荐使用 Gram-Schmidt 正交化, 而 Oxberry 等人在 [13] 中选择使用薄 QR 分解。

例 1 中的数值实验表明, 上述两项占用了 CPU 时间的大部分。

为了避免巨大的计算成本, Brand [4] 建议间接更新矩阵 Q 和 R 。他的想法是将 SVD 分解为五个矩阵

$$U = Q_{\text{old}}\tilde{Q}_{\text{old}}\Sigma_{\text{old}}\tilde{R}_{\text{old}}^\top R_{\text{old}}^\top$$

其中 $Q_{\text{old}}\tilde{Q}_{\text{old}}$ 、 $R_{\text{old}}\tilde{R}_{\text{old}}$ 、 Q_{old} 和 $\tilde{Q}_{\text{old}}$ (但不是 $\tilde{R}_{\text{old}}$ 或 $R_{\text{old}}^\top$) 是正交的。大的外部矩阵仅记录左右子空间的跨度, 并通过向 Q_{old} 追加列和向 R_{old} 追加行来构建。更新仅涉及小矩阵 $\tilde{Q}_{\text{old}}$ 和 $\tilde{R}_{\text{old}}$ 。这使得更新更快。

然而, 经过数千或数百万次更新, 乘法可能会因数值误差而侵蚀正交性; 见图 3 中的右图。因此, 需要对这些小正交矩阵进行重正交化。在 [4]

中, Brand 问道: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度。” 我们还注意到, 对于某些数据集, 仅对小矩阵 $\tilde{Q}$ 进行重正交化是不够的。在例 2 中, 我们表明, 如果不对外部大矩阵 Q_{old} 进行重正交化, 算法会提供不真实的 SVD。

在本文中, 我们首先回答这个开放问题, 答案是我们可以避免计算大量这些正交矩阵, 因此通过修改 Brand 的算法, 对一些小正交矩阵的重正交化不是必需的。其次, 我们为外部大矩阵 Q_{old} 提出了一种有效的重正交化方法, 例 2 和例 3 中的数值实验表明, 该重正交化方法在实践中效果良好。第三, 我们证明了我们的修改不会改变 Brand 算法的输出。

2 增量 SVD (I)

我们首先介绍本文所需的符号。令 I_k 为 $k \times k$ 单位矩阵, 并定义 W -加权内积:

$$(a, b)_W = a^\top W b, \quad a, b \in \mathbb{R}^m.$$

设 U 为 $m \times n$ 稠密矩阵, u_j 表示 U 的第 j 列, 即

$$U = [u_1 \mid u_2 \mid \cdots \mid u_n].$$

为方便起见, 本文采用 Matlab 符号。我们使用 $U(:, 1:\ell)$ 和 U_ℓ 表示 U 的前 ℓ 列, 使用 $U(1:\ell, 1:\ell)$ 表示 U 的 ℓ 阶首主子式。函数 $\text{diag}: \mathbb{R}^k \rightarrow \mathbb{R}^{k \times k}$ 输入一个向量, 输出一个对角矩阵, 该向量的条目在其主对角线上, 如下所示: 如果 $\alpha = (a_1, a_2, \dots, a_k)^\top$, 则

如果数据来自 PDE 的 Galerkin 型模拟, 直接应用 Brand 算法时, 需要处理加权矩阵的 Cholesky 分解。为避免这种情况, Fareed 等人 [7] 扩展了 Brand 算法以处理此类数据, 而无需计算 Cholesky 分解。更具体地说, 如果数据位于有限元空间中, 并使用一组基函数表示。那么数据的 SVD 等价于找到其系数矩阵的所谓核心 SVD。

定义 1. [7] 矩阵 $U \in \mathbb{R}^{m \times n}$ 的核心 SVD 是一种分解 $U = Q\Sigma R^\top$, 其中 $Q \in \mathbb{R}^{m \times d}$, $\Sigma \in \mathbb{R}^{d \times d}$, 且 $R \in \mathbb{R}^{n \times d}$ 满足

$$Q^\top W Q = I, \quad R^\top R = I, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_d),$$

其中 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d > 0$ 。值 $\{\sigma_i\}$ 称为 U 的 (正) 奇异值, Q 和 R 的列称为 U 的相应奇异向量。

我们注意到, 如果 $W = I$, 则上述核心 SVD 简化为矩阵 U 的标准 SVD。在 [7] 中, W 是标准有限元质量矩阵或刚度矩阵。关于加权范数设置下增量 SVD 的更多信息, 请参见 [5,6]。

接下来, 我们介绍 Brand 的增量 SVD 算法, 并遵循 [7] 中的讨论。

步骤 1: 初始化。假设 U 的第一列非零, 即 $u_1 \neq 0$, 我们通过设置下式初始化 u_1 的核心 SVD

$$\Sigma = (u_1^\top W u_1)^{1/2}, \quad Q = u_1 \Sigma^{-1}, \quad R = 1.$$

该算法如算法 1 所示。

步骤 2: U_ℓ 的核心 SVD。假设我们已经有了 U_ℓ 的秩为 k 的截断核心 SVD:

$$U_\ell = Q \Sigma R^\top, \quad \text{with} \quad Q^\top W Q = I_k, \quad R^\top R = I_k, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_k), \quad (2.1)$$

其中 $\Sigma \in \mathbb{R}^{k \times k}$ 是对角矩阵, 对角线上是 U_ℓ 的 k 个 (有序) 奇异值, $Q \in \mathbb{R}^{m \times k}$ 是包含 U_ℓ 对应 k 个左奇异向量的矩阵, $R \in \mathbb{R}^{\ell \times k}$ 是包含 U_ℓ 对应 k 个右奇异向量的矩阵。

算法 1 (初始化增量 SVD)

输入: $u_1 \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$

1: $\Sigma = (u_1^\top W u_1)^{1/2}$; $Q = u_1 \Sigma^{-1}$; $R = 1$;

2: return Q, Σ, R

步骤 3: 更新 $U_{\ell+1}$ 的核心 SVD。我们下一步的目标是更新上述 SVD, 即我们希望通过使用 U_ℓ 的核心 SVD 和新添加的向量 $u_{\ell+1}$ 来找到 $U_{\ell+1}$ 的核心 SVD。

为此, 令 $e \in \mathbb{R}^m$ 为 $u_{\ell+1}$ 在由 Q 的列张成的空间上关于加权内积 $(\cdot, \cdot)_W$ 的残差。因此,

$$e = u_{\ell+1} - Q Q^\top W u_{\ell+1}.$$

令 p 为 e 的模, 即 $p = (e^\top W e)^{1/2}$ 。如果 $p > 0$, 我们令 $\tilde{e}$ 为 e 方向上的单位向量, 即 $\tilde{e} = e/p$ 。如果 $p = 0$, 我们设 $\tilde{e} = 0$ 。那么我们有基本恒等式:

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = [Q\Sigma R^\top \mid u_{\ell+1}] = [Q \mid \tilde{e}] \left[\underbrace{\Sigma}_0 \underbrace{Q^\top W u_{\ell+1}}_p \right] \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (2.2)$$

我们可以通过找到上述恒等式右侧矩阵 Y 的完全 SVD 来找到更新后矩阵 $U_{\ell+1}$ 的 SVD。

(1) 如果 $p < \text{tol}$, 我们近似并设 $p = 0$ 和 $\tilde{e} = 0$ 。令 $Q_Y \Sigma_Y R_Y^\top$ 为 $Y = \begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & 0 \end{bmatrix}$ 的完全 SVD。那么 $U_{\ell+1} \in \mathbb{R}^{m \times (\ell+1)}$ 的核心 SVD 由下式给出

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = ([Q \mid 0]Q_Y)\Sigma_Y \left(\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y \right)^\top.$$

这表明了以下更新

$$Q \leftarrow Q Q_Y(1:k, 1:k), \quad \Sigma \leftarrow \Sigma_Y(1:k, 1:k), \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(\cdot, 1:k). \quad (2.3)$$

(2) 如果 $p \geq \text{tol}$, 我们令 $Q_Y \Sigma_Y R_Y^\top$ 为 (2.2) 中中间矩阵 Y 的完全 SVD, 然后通过下式更新 $U_{\ell+1}$ 的 SVD

$$Q \leftarrow [Q \mid \tilde{e}]Q_Y \quad \Sigma \leftarrow \Sigma_Y \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y. \quad (2.4)$$

在 [7] 中, 作者考虑了来自二维圆柱绕流模拟的数据。增量 SVD 算法持续计算非常小的奇异值 (接近机器精度) ——这需要巨大的计算成本, 并且这些值在它们的应用中并不需要。因此, 他们在 [7, 算法 4] 中建议, 如果最后几个奇异值小于容差, 则进行另一种截断。即, 找到最小的 r 使得 $\Sigma_Y(r, r) \geq \text{tol}$ 且 $\Sigma_Y(r+1, r+1) < \text{tol}$ (原文此处可能有笔误, 应为 $\Sigma_Y(r+1, r+1) < \text{tol}$ 以确定截断位置), 并设置以下截断:

$$Q \leftarrow Q(\cdot, 1:r), \quad \Sigma \leftarrow \Sigma(1:r, 1:r), \quad R \leftarrow R(\cdot, 1:r), \quad (2.5)$$

其中 Q, Σ 和 R 在 (2.4) 中定义。现在我们将上述讨论总结在算法 2 中。

算法 2 (更新增量SVD (I))

输入: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{k \times k}$, $u_{+1} \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$, tol

```

1: 设  $d = Q^T (W u_{+1})$ ;  $e = u_{+1} - Q d$ ;  $p = (e^T W e)^{1/2}$ ;
2: if  $p < \text{tol}$  then
3:   设  $p = 0$ ;
4: else
5:   设  $e = e / p$ ;
6: end if
7: 设  $Y = [\Sigma, d; 0, p]$ ;
8:  $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$ ;
9: if  $p < \text{tol}$  then
10:   设  $Q = Q Q_Y(1:k, 1:k)$ ,  $\Sigma = \Sigma_Y(1:k, 1:k)$ ,  $R = [R, 0; 0, 1] R_Y(:, 1:k)$ ;
11: else
12:   设  $Q = [Q, e] Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R = [R, 0; 0, 1] R_Y$ .
13:   if  $\Sigma(r, r) \geq \text{tol} \ \&\& \ \Sigma(r+1, r+1) < \text{tol}$  then % 根据描述调整截断条件
14:     设  $Q = Q(:, 1:r)$ ,  $\Sigma = \Sigma(1:r, 1:r)$ ,  $R = R(:, 1:r)$ ;
15:   end if
16: end if
17: return  $Q, \Sigma, R$ 

```

接下来, 我们讨论 Brandt 算法 (算法 2) 在 $W = I$ 时的复杂度, 并注意 $U \in \mathbb{R}^{m \times n}$ 具有低秩 r 。总时间复杂度为 $\mathcal{O}((m+n)r^2)$, 细节如下所示。

1. 第 1 行: 需要执行 n 次, 每次的计算成本约为 $\mathcal{O}(mr)$, 总复杂度为 $\mathcal{O}(mnr)$ 。2. 第 8 行: 由于 Y 是一个单列边界的对角矩阵, 因此可以在 $\mathcal{O}(r^2)$ 时间内进行双对角化, 总时间为 $\mathcal{O}(nr^2)$ 。3. 第 10-12 行: 每次的计算成本约为 $\mathcal{O}(mr^2 + \ell r^2)$, 因此总复杂度约为 $\mathcal{O}(\sum_{\ell=1}^n (mr^2 + \ell r^2)) = \mathcal{O}((m+n)nr^2)$ 。

步骤 4: 重正交化。理论上, 上述 SVD 更新产生正交的左右奇异向量。然而, 在实践中, 微小的数值误差会导致正交性损失。那么增量 SVD 算法就是一个非正交分解, 奇异值不真实, 因此截断不可靠。因此需要重正交化, 如果矩阵的秩不是非常低, 计算成本可能很高。具体来说, Brand 推荐使用 Gram-Schmidt 正交化 [3], 而 Oxberry 等人在 [13] 中选择使用薄 QR 分解。我们的数值测试表明, 当加权矩阵是单位矩阵时, 薄 QR 分解比 Gram-Schmidt 更快, 参见例 1。但是, 由于目前没有适用于加权范数设置的薄 QR 分解, Fareed 等人在 [7] 中使用 W -加权 Gram-Schmidt 进行重

正交化。

算法 3 (重正交化)

输入: $Q \in \mathbb{R}^{m \times k}$, $W \in \mathbb{R}^{m \times m}$, tol

```
1: if  $|(Q(:, \text{end}))^T W Q(:, 1)| > \text{tol}$  then
2:   for  $i = 1$  to  $k$  do
3:     设  $q = Q(:, i)$ 
4:     for  $j = 1$  to  $i-1$  do
5:        $q(:, i) = q(:, i) - (q^T W Q(:, j)) Q(:, j)$ 
6:     end for
7:     设  $\text{norm} = (q^T W q)^{1/2}$ 
8:     设  $Q(:, i) = q / \text{norm}$ 
9:   end for
10: end if
11: return  $Q$ 
```

我们不讨论重正交化的时间复杂度, 因为不清楚需要执行算法 3 多少次。然而, 例 1 中的数值实验表明, 上述正交化步骤是增量 SVD 算法计算成本的一大部分。

最后, 我们在算法 4 中总结了带有 W -加权 Gram-Schmidt 重正交化的增量 SVD 的完整实现。

算法 4 (完全增量 SVD (I))

输入: $W \in \mathbb{R}^{m \times m}$, tol

```
1: 获取  $u_1$ 
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$  % 算法 1
3: for  $i = 2, \dots, n$  do
4:   获取  $u_i$ 
5:    $[Q, \Sigma, R] = \text{UpdateISVD1}(Q, \Sigma, R, u_i, W, \text{tol})$  % 算法 2
6:    $Q = \text{Reorthogonalization}(Q, W, \text{tol})$  % 算法 3
7: end for
8: return  $Q, \Sigma, R$ 
```

例 1. 在此示例中, 我们测试算法 4。令 $\Omega \subset \mathbb{R}^2$ 为单位正方形, 我们将其划分为 524288 个均匀三角形。令 $\{\phi_i\}_{i=1}^m$ 为标准线性有限元基函数。接

下来, 令 $\{(x_i, y_j)\}_{i,j=1}^m$ 为 Ω 的网格点, $\{t_i\}_{i=1}^n$ 为 $[0, 10]$ 上的等距网格, 时间步长 $\Delta t = 1/10^3$ 。定义

$$f(t, x, y) = \cos(t(x + y)), \quad b_i = [(f(t_i), \phi_j)]_{j=1}^m, \quad B = [b_1 | b_2 | \dots | b_n],$$

$$M = [(\phi_j, \phi_i)]_{i,j=1}^m, \quad u_k = [f(t_k, x_i, y_j)]_{i,j=1}^m, \quad U = [u_1 | u_2 | \dots | u_n].$$

这里 M 是有限元质量矩阵, u_k 是 $f(t, x, y)$ 在 $t = t_k$ 时的线性拉格朗日插值的系数。我们通过设置 $\text{tol} = 10^{-12}$ 计算矩阵 B 在 $W = I$ 下的 SVD 和矩阵 U 在 $W = M$ 下的 SVD。令 $\|A\|_F$ 为矩阵 A 的 Frobenius 范数, 我们使用

$$\mathcal{E}_W = \|I - Q^\top W Q\|$$

来度量正交矩阵 Q 在加权内积 $(\cdot, \cdot)_W$ 下的正交性误差。

在图 1 中, 我们展示了左奇异矩阵 Q 在不同加权范数下的正交性。对于 $W = M$ 的情况, Q 的正交性被侵蚀 (即使进行了重正交化)。

图 1: 例 1: 左: $W = I$ 。右: $W = M$ 。

整个模拟对于 $W = I$ 和 $W = M$ 分别花费约 154 秒和 4000 秒。接下来, 我们在表 1 中展示了算法 4 主要部分的 CPU 时间 (秒)。我们看到, 正交化步骤是增量 SVD 算法计算成本的一大部分, 尤其是在 $W = M$ 的情况下。

表 1: 不同加权内积下的 CPU 时间 (秒)。

	算法 2 第 1 行	算法 2 第 9-13 行	算法 3
W = I	32.03	460.21	360.13
W = M	110.58	115.96	3761.6

3 增量 SVD (II)

算法 4 易于实现, 但计算成本高, 特别是在加权范数设置下。对于一个 $m \times n$ 的低秩 r 稠密矩阵, 算法 4 的时间复杂度为 $\mathcal{O}(m(m+n)r^2)$ 。在 [4] 中, Brand 将复杂度降低到 $\mathcal{O}(mnr)$ 。然而, 为了更新左奇异空间, 必须计算数千或数百万个正交矩阵并将它们相乘。更新右奇异空间更为复杂, 需要计算数千或数百万次矩阵的伪逆。

接下来, 我们遵循 [4] 将 Brand 的改进扩展到计算关于加权内积 $(\cdot, \cdot)_W$ 的矩阵 $U \in \mathbb{R}^{m \times n}$ 的核心 SVD。我们不按照 (2.3) 或 (2.4) 中规定的那样更新大的奇异向量矩阵, 而是假设 $U_\ell \in \mathbb{R}^{m \times \ell}$ 的核心 SVD 可以写成以下五个矩阵

$$U_\ell = Q_{\text{old}} \tilde{Q}_{\text{old}} \Sigma_{\text{old}} \tilde{R}_{\text{old}}^\top R_{\text{old}}^\top. \quad (3.1)$$

这里 $Q_{\text{old}} \in \mathbb{R}^{m \times k}$ 是在加权内积下正交的矩阵, $\tilde{Q}_{\text{old}}$ 和 $\tilde{R}_{\text{old}}^\top R_{\text{old}}^\top$ 是标准正交矩阵, 但 $\tilde{R}_{\text{old}}^\top$ 和 $R_{\text{old}}^\top$ 不是。大的外部矩阵仅记录左右子空间的跨度, 并通过向 Q_{old} 追加列和向 $R_{\text{old}}^\top$ 追加行来构建。并且更新仅涉及小矩阵 $\tilde{Q}_{\text{old}}$ 和 $\tilde{R}_{\text{old}}^\top$, 这使得更新更快。

接下来, 我们考虑 $U_{\ell+1} = [U_\ell \mid u_{\ell+1}] \in \mathbb{R}^{m \times (\ell+1)}$ 的核心 SVD。类似于第 2 节, 令 $e \in \mathbb{R}^m$ 为 $u_{\ell+1}$ 在由 $Q_{\text{old}} \tilde{Q}_{\text{old}}$ 的列张成的空间上关于加权内积 $(\cdot, \cdot)_W$ 的残差, 即

$$e = u_{\ell+1} - (Q_{\text{old}} \tilde{Q}_{\text{old}})(Q_{\text{old}} \tilde{Q}_{\text{old}})^\top W u_{\ell+1} = u_{\ell+1} - Q_{\text{old}} Q_{\text{old}}^\top W u_{\ell+1}.$$

令 $p = (e^\top W e)^{1/2}$ 。如果 $p > 0$, 我们令 $\tilde{e} = e/p$, 否则设 $\tilde{e} = 0$ 。然后由 (3.1) 和基本恒等式 (2.2) 我们有

$$U_{\ell+1} = \left[Q_{\text{old}} \tilde{Q}_{\text{old}} \mid \tilde{e} \right] \underbrace{\begin{bmatrix} \Sigma_{\text{old}} & \tilde{Q}_{\text{old}} Q_{\text{old}}^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_Y \begin{bmatrix} R_{\text{old}} \tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix}^\top.$$

假设 $Q_Y \Sigma_Y R_Y^\top$ 是 Y 的完全 SVD, 那么 $U_{\ell+1}$ 的核心 SVD 由下式给出

$$U_{\ell+1} = \left(\left[Q_{\text{old}} \tilde{Q}_{\text{old}} \mid \tilde{e} \right] Q_Y \right) \Sigma_Y \left(\begin{bmatrix} R_{\text{old}} \tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} R_Y \right)^\top. \quad (3.2)$$

(1) 如果 $p < \text{tol}$, 我们设 $p = 0$ 和 $\tilde{e} = 0$, 则更新不会增加秩。我们通过下式更新小正交矩阵 $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times k}$ 和对角矩阵 $\Sigma_{\text{new}} \in \mathbb{R}^{k \times k}$

$$\tilde{Q}_{\text{new}} \leftarrow \tilde{Q}_{\text{old}} Q_Y(1:k, 1:k), \quad \Sigma_{\text{new}} \leftarrow \Sigma_Y(1:k, 1:k). \quad (3.3)$$

(2) 如果 $p \geq \text{tol}$, 我们然后将向量 $\tilde{e}$ 添加到 Q_{old} 来更新大的外部矩阵 Q_{new} , 并通过下式更新小正交矩阵 $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times (k+1)}$ 和对角矩阵 $\Sigma_{\text{new}} \in \mathbb{R}^{(k+1) \times (k+1)}$

$$Q_{\text{new}} \leftarrow [Q_{\text{old}} \mid \tilde{e}], \quad \tilde{Q}_{\text{new}} \leftarrow \begin{bmatrix} \tilde{Q}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} Q_Y, \quad \Sigma_{\text{new}} \leftarrow \Sigma_Y. \quad (3.4)$$

(3.3) 或 (3.4) 的时间复杂度约为 $\mathcal{O}(r^2)$, 因为左奇异向量矩阵的更新是通过两个小正交矩阵的乘法完成的。这与 (2.3) 或 (2.4) 相比是一个主要区别。

然而, 我们必须在数千或数百万次更新中更新小正交矩阵, 乘法可能会因数值误差而侵蚀 $\tilde{Q}$ 的正交性。因此, 当 $\tilde{Q}$ 的第一个和最后一个左奇异向量之间的内积大于某个容差时, 需要进行重正交化。

在 [4] 中, Brand 问道: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度”。在下一节中, 我们回答这个问题, 答案是我们 ** 可以避免 ** 计算大量那些正交矩阵, 并且这些乘法不是必需的。因此, 不需要重正交化。

1. 理论上, 新添加的向量 $\tilde{e}$ 与 Q_{old} 的每一列正交。然而, 在实践中, 对于某些数据集, 矩阵 $[Q_{\text{old}} \mid \tilde{e}]$ 非常快地失去正交性。因此需要对矩阵进行重正交化。我们注意到, 在 [4] 中, Brand 没有包括这一点。在例 2 中, 我们展示了这有多重要, 并在下一节中提出了一种非常有效的方法来解决这个问题。

右奇异向量矩阵 R_{new} 和 $\tilde{R}_{\text{new}}$ 的更新更为复杂, 因为需要在保证乘积 $R_{\text{new}}\tilde{R}_{\text{new}}$ 的列是正交的同时向 R_{old} 添加行。从基本恒等式 (2.2) 和 (3.2) 可知, 右侧更新必须满足

在以下更新中, 我们需要计算 $\tilde{R}_{\text{old}}$ 的伪逆和 R_Y 的子矩阵。我们用 X^+ 表示 X 的伪逆。

- (3) 如果 $p < \text{tol}$, 秩不增加, R_Y 的最后一列表示一个未使用的子空间维度, 应被抑制。令 $\mathcal{R}_1 = R_Y(1:k, 1:k)$, $\mathcal{R}_2 = R_Y(k+1, 1:k)$,

$$\tilde{R}_{\text{new}} \leftarrow \tilde{R}_{\text{old}}\mathcal{R}_1, \quad \tilde{R}_{\text{new}}^+ \leftarrow \mathcal{R}_1^+ \tilde{R}_{\text{old}}^+, \quad R_{\text{new}} \leftarrow \begin{bmatrix} R_{\text{old}} & \mathcal{R}_2 \tilde{R}_{\text{new}}^+ \end{bmatrix}. \quad (3.5)$$

- (4) 如果 $p \geq \text{tol}$, 更新是秩增加的。我们通过下式更新小正交矩阵 $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times k}$ 和对角矩阵 $\Sigma_{\text{new}} \in \mathbb{R}^{k \times k}$

$$\tilde{R}_{\text{new}} \leftarrow \begin{bmatrix} \tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} R_Y, \quad \tilde{R}_{\text{new}}^+ \leftarrow R_Y^+ \begin{bmatrix} \tilde{R}_{\text{old}}^+ & 0 \\ 0 & 1 \end{bmatrix}, \quad R_{\text{new}} \leftarrow \begin{bmatrix} R_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix}. \quad (3.6)$$

该增量 SVD 算法的实现如算法 5 和 6 所示。

图 2: B 的奇异值: 左: 算法 6。右: MATLAB 内置函数 `svd()`。

接下来, 我们向算法 6 添加重正交化 (细节将在下一节给出), 并绘制修改后的算法 6 与 MATLAB 内置函数 `svd()` 之间的误差 (图 3 左)。我们看到新添加的重正交化在实践中效果良好。

最后, 我们测试每次更新时小正交矩阵 $\tilde{Q}$ 的正交性, 我们使用 $\mathcal{E}_I(\tilde{Q}) = \|I - \tilde{Q}^\top \tilde{Q}\|$ 来度量 $\tilde{Q}$ 的正交性, 如图 3 右所示, 我们看到即使在执行重正交化之后, 矩阵 $\tilde{Q}$ 在 $n = 600$ 后也很快失去正交性。

图 3: 例 2: 左: 通过添加重正交化后, B 的前 34 个奇异值的误差。右: 每次更新时 $\tilde{Q}$ 的正交性误差。

4 我们的改进

正如第 3 节所指出的, 算法 6 需要在数千或数百万次更新中更新小正交矩阵 $\tilde{Q}$, 乘法可能会因数值误差而侵蚀 $\tilde{Q}$ 的正交性。为了克服这个问题, Brand 提出当 $\tilde{Q}$ 的第一个和最后一个左奇异向量之间的内积大于某个容差时进行重正交化。然而, 正如我们在例 2 中看到的, 即使我们执行了重正交化, 正交矩阵 $\tilde{Q}$ 仍然可能失去正交性。此外, 为了更新右奇异空间, 算法 6 需要计算数千或数百万次矩阵的伪逆。

首先, 我们总结一下本节的贡献。

(1) 我们回答了 Brand 在 [4] 中提出的问题: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度”。(2) 我们证明了新算法的输出与 Brand 的算法相同。(3) 我们表明, 不需要计算数千或数百万次矩阵的伪逆来更新右奇异子空间。(4) 我们为矩阵 $[Q_{\text{old}} | \tilde{e}]$ 提出了一种重正交化方法, 我们注意到这一步在 [4] 的原始算法中是没有的。在例 2 中我们已经证明了重正交化是必要的。

首先, 我们引入以下引理:

引理 1. 设 $A \in \mathbb{R}^{k \times k}$, $\alpha \in \mathbb{R}^k$, 并令 $Q_1 \Sigma_1 R_1^\top$ 为 $[A | \alpha]$ 的薄 SVD, 那么

$$\begin{bmatrix} Q_1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} R_1^\top \\ r_{k+1}^\top \end{bmatrix} \quad (4.1)$$

是 $\begin{bmatrix} A & \alpha \\ 0 & 0 \end{bmatrix}$ 的一个 SVD，其中 $r_{k+1} \in \mathbb{R}^{k+1}$ 是一个单位向量，并且与 R_1 的每一列正交。

接下来，我们使用引理 1 来修改秩不增加时左奇异子空间的更新。假设 $Q\Sigma R^\top$ 是 U_ℓ 的秩为 k 的截断核心 SVD， $u_{\ell+1} - QQ^\top W u_{\ell+1}$ 的范数非常小，我们将其近似为 0。因此，

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = Q [\Sigma \mid Q^\top W u_{\ell+1}] \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (4.2)$$

令 $Q_1 \Sigma_1 R_1^\top$ 为 $[\Sigma \mid Q^\top W u_{\ell+1}]$ 的薄 SVD，那么 $U_{\ell+1}$ 的核心 SVD 由下式给出

$$U_{\ell+1} = QQ_1 \Sigma_1 R_1^\top \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (4.3)$$

因此，以下更新

$$Q \leftarrow QQ_1, \quad \Sigma = \Sigma_1, \quad R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_1, \quad (4.4)$$

由于引理 1，与 (2.3) 完全相同。

接下来，我们考虑一个更一般的情况。假设矩阵 C 有 s 列，并且 $C - QQ^\top WC$ 的范数为零或非常小，那么我们将其近似为 0，然后有以下基本恒等式：

$$U_{\ell+1} = [U_\ell \mid C] = Q [\Sigma \mid Q^\top WC] \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix}^\top. \quad (4.5)$$

在本文中，我们使用 MATLAB 内置函数 `svd(Y, 'econ')` 来找到 Y 的 SVD，因为矩阵 Y 总是短而胖的。

4.1 更新左右奇异空间

由于我们假设矩阵 U 是低秩的,那么可以合理地预期 $\{u_{\ell+1}, u_{\ell+2}, \dots, u_n\}$ 中的大多数向量与 $Q \in \mathbb{R}^{m \times k}$ 中的向量线性相关或几乎线性相关。不失一般性,我们假设接下来的两个向量 $u_{\ell+1}$ 和 $u_{\ell+2}$, 当将它们投影到由 Q 的列张成的空间上时, 它们的残差小于算法 4 中的 tol 。换句话说, 我们假设

$$\|u_{\ell+1} - QQ^\top W u_{\ell+1}\|_W < \text{tol}, \quad \|u_{\ell+2} - QQ^\top W u_{\ell+2}\|_W < \text{tol}. \quad (4.6)$$

然后我们按照 Brand 的算法增量地找到 $U_{\ell+2}$ 的 SVD:

(1) 找到 $[\Sigma \mid Q^\top W u_{\ell+1}]$ 的 SVD, 并假设

$$[\Sigma \mid Q^\top W u_{\ell+1}] = Q_{(1)} \Sigma_{(1)} R_{(1)}^\top, \quad (4.7)$$

$$Q_{(1)} \in \mathbb{R}^{k \times k}, \Sigma_{(1)} \in \mathbb{R}^{k \times k}, R_{(1)} \in \mathbb{R}^{(k+1) \times k} \quad \text{且} \quad Q_{(1)}^\top Q_{(1)} = Q_{(1)} Q_{(1)}^\top = I_k.$$

(2) 通过下式更新 $U_{\ell+1}$ 的 SVD

$$Q_{\ell+1} = QQ_{(1)}, \quad \Sigma_{\ell+1} = \Sigma_{(1)}, \quad R_{\ell+1} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_{(1)}. \quad (4.8)$$

根据以上讨论, 更新 (4.8) 与 (2.3) 完全相同。接下来, 根据算法 4, 我们应该检查当 $u_{\ell+2}$ 投影到由 $Q_{\ell+1}$ 的列张成的空间上时, 其残差的大小。为此, 我们引入以下引理。

引理 2. 令 $Q \in \mathbb{R}^{k \times k}$ 为一个正交矩阵, 则对于任意 $x \in \mathbb{R}^m$ 和 $Q \in \mathbb{R}^{m \times k}$, 我们有

$$QQ^\top x = (QQ)(QQ)^\top x.$$

由于 $Q_{(1)}$ 是一个正交矩阵, 那么根据引理 2 和 (4.6), 我们有

$$\|u_{\ell+2} - Q_{\ell+1} Q_{\ell+1}^\top W u_{\ell+2}\|_W = \|u_{\ell+2} - QQ^\top W u_{\ell+2}\|_W < \text{tol}.$$

接下来, 我们按照与上述步骤 (1) 和 (2) 相同的论点来处理 $u_{\ell+2}$ 。

(3) 找到 $[\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2}]$ 的 SVD, 并假设

$$[\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2}] = Q_{(2)} \Sigma_{(2)} R_{(2)}^\top, \quad (4.9)$$

$$Q_{(2)} \in \mathbb{R}^{k \times k}, \Sigma_{(2)} \in \mathbb{R}^{k \times k}, R_{(2)} \in \mathbb{R}^{(k+1) \times k} \quad \text{且} \quad Q_{(2)}^\top Q_{(2)} = Q_{(2)} Q_{(2)}^\top = I_k.$$

根据 4.8, 我们通过下式更新 $U_{\ell+2}$ 的 SVD

$$\begin{aligned} Q_{\ell+2} &= Q_{\ell+1} Q_{(2)} = Q Q_{(1)} Q_{(2)}, \\ \Sigma_{\ell+2} &= \Sigma_{(2)}, \\ R_{\ell+2} &= \begin{bmatrix} R_{\ell+1} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}. \end{aligned} \quad (4.10)$$

算法 4 通过首先计算 $Q_{\ell+1} = Q Q_{(1)}$, 然后计算 $Q_{\ell+2} = Q_{\ell+1} Q_{(2)}$ 来更新左奇异矩阵 $Q_{\ell+2}$ 。这种更新的缺点是计算成本高。为了避免这种情况, Brand 提出首先计算 $\tilde{Q} = Q_{(1)} Q_{(2)}$, 然后通过计算 $Q \tilde{Q}$ 来更新矩阵 Q ; 详见算法 6。对于右奇异空间, 算法 4 首先计算 $R_{\ell+1} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_{(1)}$, 然后

计算 $R_{\ell+2} = \begin{bmatrix} R_{\ell+1} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$ 来更新 $R_{\ell+2}$ 。为了节省 CPU 时间, 可以使用 Brand 的算法来更新右奇异子空间, 但必须计算 $R_{(1)}$ 和 $R_{(2)}$ 的子矩阵的伪逆。

从例 1 和例 2 中, 我们已经看到两种算法在左奇异向量矩阵的正交性损失方面都受到了影响。

下面的新算法可以在不改变算法 6 的复杂度的情况下避免上述问题。为了描述我们的新算法, 我们需要以下基本恒等式。

$$\begin{bmatrix} A & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} B & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} A & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} B & 0 \\ 0 & 1 \end{bmatrix}, \quad A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{(n+1) \times n}. \quad (4.11)$$

根据基本恒等式 (4.11), 我们可以将 $R_{\ell+2}$ 重写为

$$R_{\ell+2} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}.$$

由 (4.10) 可知, $U_{\ell+2}$ 的 SVD 由下式给出

$$U_{\ell+2} = Q Q_{(1)} Q_{(2)} \Sigma_{(2)} R_{(2)}^\top \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix}^\top. \quad (4.12)$$

受 (4.12) 的启发, 如果我们能直接计算 $Q_{(1)} Q_{(2)}$ 和 $\begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$, 那么 $U_{\ell+2}$ 的 SVD 更新可能会避免许多矩阵-矩阵乘法操作。

定理 1. 令 $\tilde{Q} = Q_{(1)} Q_{(2)}$, $\tilde{\Sigma} = \Sigma_{(2)}$ 和 $\tilde{R} = \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$, 则 $\tilde{Q} \tilde{\Sigma} \tilde{R}^\top$ 是

$$[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}]$$

的一个薄 SVD。

证明。首先, 使用 (4.7) 我们有

$$[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] = [Q_{(1)} \Sigma_{(1)} R_{(1)}^\top \mid Q^\top W u_{\ell+2}].$$

注意 $Q_{(1)}^\top Q_{(1)} = Q_{(1)} Q_{(1)}^\top = I_k$, 并且根据基本恒等式 (4.5) 我们有

$$\begin{aligned} [\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] &= Q_{(1)} [\Sigma_{(1)} \mid Q_{(1)}^\top Q^\top W u_{\ell+2}] \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \\ &= Q_{(1)} [\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2}] \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top, \end{aligned}$$

其中我们使用了 (4.8) 中的前两个恒等式。接下来我们使用 (4.9) 得到:

$$\begin{aligned} [\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] &= Q_{(1)} Q_{(2)} \Sigma_{(2)} R_{(2)}^\top \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \\ &= \tilde{Q} \tilde{\Sigma} \tilde{R}^\top. \quad \blacksquare \end{aligned}$$

定理 1 表明, 我们可以通过首先计算 $[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] \tilde{Q} \tilde{\Sigma} \tilde{R}^\top$ 的薄 SVD, 然后通过下式更新它们来更新 $U_{\ell+2}$ 的 SVD

$$Q_{\ell+2} = Q\tilde{Q}, \quad \Sigma_{\ell+2} = \tilde{\Sigma}, \quad R_{\ell+2} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \tilde{R}. \quad (4.13)$$

现在我们将上述讨论扩展到更一般的情况。

定理 2. 令 $Q\Sigma R^\top$ 为 U_ℓ 的秩为 k 的截断核心 SVD, $u_{\ell+j}$ 为 U 的第 $(\ell+j)$ 列, $j = 1, 2, \dots, s$ 。如果当将它们投影到由 Q 的列张成的空间上时, 它们的残差小于算法 4 中的 tol 。令 $\tilde{Q}\tilde{\Sigma}\tilde{R}^\top$ 为

$$[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2} \mid \dots \mid Q^\top W u_{\ell+s}] \in \mathbb{R}^{k \times (k+s)} \quad (4.14)$$

的标准薄 SVD。那么我们建议进行以下更新

$$Q \leftarrow Q\tilde{Q}, \quad \Sigma \leftarrow \tilde{\Sigma}, \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix} \tilde{R}. \quad (4.15)$$

此外, (4.15) 中的更新与算法 4 的输出完全相同。

注 2. 在实际计算中, 我们不计算 $Q\tilde{Q}$, 只需要存储 Q 和 $\tilde{Q}$ 以备将来更新。由于我们只需要计算 $\tilde{Q} r$ 次, 那么它们的乘积不会失去正交性。因此, 我们对 Brand 在 [4] 中的问题的回答是: 对 $\tilde{Q}$ 的重正交化是不需要的。此外, 为了更新右奇异子空间, 我们不计算矩阵的伪逆。值得提及的是, 在实践中, 我们不通过将 $\tilde{R}$ 分割成 $\begin{bmatrix} \tilde{R}_1 \\ \tilde{R}_2 \end{bmatrix}$ 来形成矩阵 $\begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix}$, 然后 R 通过 $\begin{bmatrix} R\tilde{R}_1 \\ \tilde{R}_2 \end{bmatrix}$ 更新。更新矩阵 R 的总计算成本为 $\mathcal{O}(nr^3)$ 。由于 $r \ll m, n$, 该成本可以忽略。然而, 值得提及的是, [4] 中的技术可以通过计算 r 次小矩阵的伪逆来更新右奇异空间, 从而将计算成本降低到 $\mathcal{O}(nr^2)$ 。

一旦完成了 $U_{\ell+s}$ 的 SVD, 我们接着更新 $U_{\ell+s+1}$ 的 SVD。为了更好地描述算法。我们假设 $Q\Sigma R^\top$ 是 U_ℓ 的核心 SVD, $\hat{Q}\hat{\Sigma}\hat{R}^\top$ 是 $U_{\ell+s}$ 的核心 SVD, 那么

$$\hat{Q} = Q\tilde{Q}, \quad \hat{R} = \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix} \tilde{R}, \quad (4.16)$$

其中 $\hat{Q}$ 和 $\hat{R}$ 分别是 (4.14) 的左、右奇异矩阵。

根据我们的假设，当将 $u_{\ell+s+1}$ 投影到由 Q 的列张成的空间上时，其残差大于 tol 。为了保持相同的精度，我们应该遵循 Brand 的想法来计算将 $u_{\ell+s+1}$ 投影到由 $\hat{Q}$ 的列张成的空间上的残差。

换句话说，对于那些列，我们必须计算两次残差。因此，这些重复的计算成本可能很高。接下来，我们表明可以避免这种成本。

为此，我们遵循 Brand 的想法，通过以下方式构造 $[U_{\ell+s} \mid u_{\ell+s+1}]$ 的 SVD

(1) 令 $e = u_{\ell+s+1} - \hat{Q}\hat{Q}^\top W u_{\ell+s+1}$ ，并令 $p = \|e\|_W$ 和 $\tilde{e} = e/p$ (2) 找到 $\bar{Y} = \begin{bmatrix} \hat{\Sigma} & \hat{Q}^\top u_{\ell+s+1} \\ 0 & p \end{bmatrix}$ 的完全 SVD，并令 $\hat{Q}\Sigma\hat{R}^\top$ 为 $\bar{Y}$ 的 SVD，然后 (3) 通过下式更新 $[U_{\ell+s} \mid u_{\ell+s+1}]$ 的 SVD

$$[U_{\ell+s} \mid u_{\ell+s+1}] = \left([\hat{Q} \mid \tilde{e}] \hat{Q} \right) \Sigma \left(\begin{bmatrix} \hat{R} & 0 \\ 0 & 1 \end{bmatrix} \tilde{R} \right)^\top. \quad (4.17)$$

接下来，我们讨论上述三个步骤。首先，由于引理 2，步骤 (1) 中的计算是不必要的。其次，我们确实需要在步骤 (2) 中计算 $\hat{Q}^\top u_{\ell+s+1}$ 。然而，由于 $\hat{Q}^\top u_{\ell+s+1} = \hat{Q}^\top Q^\top u_{\ell+s+1}$ ，并且我们已经计算了 $Q^\top u_{\ell+s+1}$ ，那么更新 $\hat{Q}^\top Q^\top u_{\ell+s+1}$ 几乎不花时间。对于步骤 (3)，通过使用 (4.16) 我们有

$$\begin{aligned} [U_{\ell+s} \mid u_{\ell+s+1}] &= \left([Q\hat{Q} \mid \tilde{e}] \hat{Q} \right) \Sigma \left(\begin{bmatrix} \hat{R} & 0 \\ 0 & 1 \end{bmatrix} \tilde{R} \right)^\top \\ &= \left([Q \mid \tilde{e}] \begin{bmatrix} \hat{Q} & 0 \\ 0 & 1 \end{bmatrix} \hat{Q} \right) \Sigma \left(\begin{bmatrix} \hat{R} & 0 \\ 0 & 1 \end{bmatrix} \tilde{R} \right)^\top. \end{aligned} \quad (4.18)$$

这表明通过添加向量 $\tilde{e}$ 来更新左奇异矩阵 Q ，计算 $\begin{bmatrix} \hat{Q} & 0 \\ 0 & 1 \end{bmatrix} \hat{Q}$ 然后存储它，计算成本为 $\mathcal{O}(r^2)$ 。对于右奇异矩阵，我们直接计算它，时间复杂度为 $\mathcal{O}(nr^2)$ 。步骤 (3) 将执行 r 次，那么这一步的总 CPU 时间为 $\mathcal{O}(nr^3)$ 。我们注意到，如果使用 [4] 中的技术，通过计算 r 次矩阵的伪逆，计算成本可以降低到 $\mathcal{O}(nr^2)$ 。

4.2 重正交化

正如我们在第 2 节中声称的，实践中的微小数值误差可能导致正交性损失。在 [7,13] 中对矩阵 $\left([\hat{Q} \mid \tilde{e}] \hat{Q} \right)$ 进行重正交化是必要的，并且由于算

法 3 中的两个循环，计算成本很高。

观察恒等式 (4.18)，我们注意到正交矩阵 $\hat{Q}$ 和 $\tilde{Q}$ 很小，并且只有 r 个它们相乘，它们可以在合理的数值精度内保持正交性。因此，我们不需要对这些小的正交矩阵执行重正交化。这回答了 Brand 在 [4] 中提出的问题。

基于我们在例 2 中的数值实验。矩阵 $[Q | \tilde{e}]$ 可能会非常快地失去正交性。因此，我们考虑对矩阵 $[Q | \tilde{e}]$ 进行重正交化。我们注意到，这是必要的，并且 Brand 的原始算法中没有包括这一点。

根据 $[Q | \tilde{e}]$ 的构造，我们可以递归地对其进行重正交化：仅对新添加的向量 $\tilde{e}$ 应用 W -加权 Gram-Schmidt，因为矩阵 Q 在先前的步骤中已经过重正交化，如果 $\tilde{e}$ 与 Q 的第一列之间的加权内积大于某个容差。此外，如果 $\tilde{e} = 0$ ，那么最后一列将被截断。换句话说，我们不需要在截断部分 ($\|p\|_W < \text{tol}$) 执行重正交化；更多细节请参见算法 7。

算法 7 (增量SVD (III))

输入: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{k \times k}$, $u_{+1} \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$, tol , V , Q_0 , q

```

1: 设  $d = Q^T (W u_{+1})$ ;  $e = u_{+1} - Q d$ ;  $p = (e^T W e)^{1/2}$ 
2: if  $p < \text{tol}$  then
3:    $q = q + 1$ 
4:   设  $V\{q\} = Q_0^T d$ 
5: else
6:   if  $q > 0$  then
7:     设  $Y = [\Sigma, \text{cell2mat}(V)]$ 
8:      $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ 
9:     设  $Q_0 = Q_0 Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,  $R_2 = R_Y(k+1, 1:\text{end}-1)$ 
10:     $R = [R * R_1; R_2]$ ;
11:    设  $d = Q_Y^T d$ 
12:  end if
13:  设  $e = e / p$ 
14:  if  $|e^T W Q(:,1)| > \text{tol}$  then
15:     $e = e - Q (Q^T (W e))$ ;  $p_1 = (e^T W e)^{1/2}$ ;  $e = e / p_1$ 
16:  end if
17:  设  $Y = [\Sigma, d; 0, p]$ 
18:   $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$ 
19:  设  $Q_0 = [Q_0, 0; 0, 1] Q_Y$ ,  $Q = [Q, e]$ ,  $\Sigma = \Sigma_Y$ ,  $R = [R, 0; 0, 1] R_Y$ 

```

```

20:   设  $V = []$ ,  $q = 0$ 
21: end if
22: return  $Q, \Sigma, R, V, Q0, q$ 

```

4.3 算法 7 的复杂度

我们将自己限制在 $W = I$ 的情况来讨论算法 7。回顾 $U \in \mathbb{R}^{m \times n}$ 具有低秩 r 。

1. 第 1 行: 这与算法 5 完全相同, 复杂度为 $\mathcal{O}(mnr)$ 。2. 第 3-4 行: 向量 $d \in \mathbb{R}^k$ 且 $k \leq r$, 因此这一步在 CPU 时间和内存存储方面几乎可以忽略。Brand 的方法没有这一步。3. 第 7-11 行: 主要成本是更新 R , CPU 时间小于 (nr^3) 。如前所述, 成本可以降低到 $\mathcal{O}(nr^2)$ 。我们将其留在此处, 因为 $r \ll m$ 。4. 第 14-16 行: 重正交化的 CPU 时间每次为 $\mathcal{O}(mr)$, 这部分将执行 $\mathcal{O}(r)$ 次, 因此总复杂度为 $\mathcal{O}(mr^2)$ 。这部分在算法 5 中没有包含, 但对于某些数据集是必要的。5. 第 17-20 行: 总复杂度小于 $\mathcal{O}(nr^3)$ 。同样, 这可以通过使用 [4] 中的技术降低到 $\mathcal{O}(nr^2)$ 。

根据我们的假设, $r \ll m, n$, 因此算法 7 的 CPU 时间为 $\mathcal{O}(mnr)$ 。

我们注意到, 算法 7 的输出 V 可能不为空。这意味着算法 7 的输出不是 U 的 SVD。因此, 我们必须更新 V 中向量的 SVD。我们在算法 8 中给出了这一步的实现, 复杂度为 $\mathcal{O}(mr^2)$ 。最后, 我们在算法 9 中完成了完整的实现。

算法 8 (增量SVD (III) 最终检查)

```

输入:  $Q \in \mathbb{R}^{m \times k}$ ,  $\Sigma \in \mathbb{R}^{k \times k}$ ,  $R \in \mathbb{R}^{k \times k}$ ,  $V$ ,  $Q0$ ,  $q$ 
1: if  $q > 0$  then
2:   设  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
3:    $[Q\_Y, \Sigma\_Y, R\_Y] = \text{svd}(Y, 'econ');$ 
4:   设  $Q = Q (Q0 \ Q\_Y)$ ,  $\Sigma = \Sigma\_Y$ ,  $R1 = R\_Y(1:k, 1:\text{end}-1)$ ,
5:    $R2 = R\_Y(k+1, 1:\text{end}-1)$ ,  $R = [R \ * \ R1; R2]$ ;
6: else
7:   设  $Q = Q \ Q0$ ;
8: end if
9: return  $Q, \Sigma, R$ .

```

算法 9 (完全增量SVD (III))

```

输入:  $W \in \mathbb{R}^{m \times m}$ , tol
1: 获取  $u_1$ ;
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$ ; % 算法 1
3: 设  $V = []$ ;  $Q_0 = 1$ ;  $q = 0$ ;
4: for  $i = 2, \dots, n$  do
5:   获取  $u$ 
6:    $[q, V, Q_0, Q, \Sigma, R] = \text{UpdateISVD3}(q, V, Q_0, Q, \Sigma, R, u, W, \text{tol})$ ; % 算法 7
7: end for
8:  $[Q, \Sigma, R] = \text{UpdateISVD3check}(q, V, Q_0, Q, \Sigma, R)$ ; % 算法 8
9: return  $Q, \Sigma, R$ 

```

例 3. 在此示例中，我们重新审视例 1 以测试算法 9 的效率和准确性。我们在图 4 中展示了左奇异向量矩阵的正交性，其中

$$\mathcal{E}_W(Q) = \|I - Q^\top W Q\|, \quad \mathcal{E}_I(\tilde{Q}) = \|I - \tilde{Q}^\top \tilde{Q}\|.$$

我们看到我们的算法 9 保持了良好的正交性。

图 4: 例 3: 上图: $W = I$ 时 $\tilde{Q}$ 和 Q 的正交性。下图: $W = M$ 时 $\tilde{Q}$ 和 Q 的正交性。

接下来，我们在表 2 中展示了算法 7 主要部分的 CPU 时间（秒）。整个模拟对于 $W = I$ 和 $W = M$ 分别花费约 34 秒和 118 秒。与表 1 相比，CPU 时间大大减少，特别是在加权范数设置下 ($W \neq I$)。此外，投影部分（算法 7 第 1 行）占整个模拟时间的约 99

表 2: 例 3: 使用算法 9 时两种不同加权范数的 CPU 时间（秒）。

	算法 7 第 1 行	第 3-4 行	第 7-11 行	第 14-16 行	第 17-20 行
$W = I$	33.457	0.2416	0.0327	0.1251	0.2882
$W = M$	116.41	0.1870	0.0650	0.6526	0.7120

5 另一种截断

对于许多数据集，它们可能有大量非零奇异值，但其中大多数非常小。如果不截断这些小奇异值，增量 SVD 算法可能会持续计算它们，因此计算

成本巨大。遵循 [7, 算法 4], 如果最后几个奇异值小于容差, 我们设置另一种截断。

接下来, 我们表明, 只需检查最后一个奇异值即可。令 $Q\Sigma R^\top$ 为 U_ℓ 的核心 SVD, $e = u_{\ell+1} - QQ^\top W u_{\ell+1}$, 且 $p = \|e\|_W > \text{tol}$, 则

$$U_{\ell+1} = [Q \mid \tilde{e}] \underbrace{\begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_Y \underbrace{\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}}_Y^\top = [Q \mid \tilde{e}] Q_Y \Sigma_Y R_Y^\top \underbrace{\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}}_Y^\top.$$

接下来, 我们估计 Y 的奇异值。

引理 3. 假设 $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_k)$ 且 $\sigma_1 \geq \dots \geq \sigma_k$, 以及 $\Sigma_Y = \text{diag}(\mu_1, \dots, \mu_{k+1})$ 且 $\mu_1 \geq \dots \geq \mu_{k+1}$ 。那么我们有

$$\begin{aligned} \mu_{k+1} &\leq p, \\ \mu_{k+1} &\leq \sigma_k \leq \mu_k \leq \sigma_{k-1} \leq \dots \leq \sigma_1 \leq \mu_1. \end{aligned} \quad (5.1a)$$

证明。首先, 我们考虑 $Y^\top Y$

$$Y^\top Y = \begin{bmatrix} \Sigma^2 & \Sigma Q^\top W u_{\ell+1} \\ u_{\ell+1}^\top W Q \Sigma & p^2 + u_{\ell+1}^\top W Q Q^\top W u_{\ell+1} \end{bmatrix}.$$

通过使用 Cauchy 交错定理 [9, 定理 1] 我们有

$$\mu_{k+1} \leq \sigma_k \leq \mu_k \leq \sigma_{k-1} \leq \dots \leq \sigma_1 \leq \mu_1.$$

接下来, 我们取 $e = [0, 0, \dots, 1]$ 得到

$$p^2 = e Y Y^\top e \geq \min_{\|x\|=1} x Y Y^\top x = \lambda_{\min}(Y Y^\top) = \mu_{k+1}^2.$$

不等式 (5.1a) 表明, 无论 p 有多大, Y 的最后一个奇异值都可能非常小。也就是说, 我们为 p 设置的容差不能避免算法持续计算非常小的奇异值。因此, 如果数据有大量非常小的奇异值, 则需要进行另一种截断。不等式 (5.1b) 保证我们只有 Y 的最后一个奇异值可能小于容差。因此, 我们只需要检查最后一个。

(i) 如果 $\Sigma_Y(k+1, k+1) \geq \text{tol}$, 则

$$Q \leftarrow [Q \mid \tilde{e}] Q_Y, \quad \Sigma \leftarrow \Sigma_Y, \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y. \quad (5.2)$$

(ii) 如果 $\Sigma_Y(k+1, k+1) < \text{tol}$, 则

$$Q \leftarrow [Q \mid \tilde{e}]Q_Y(:, 1:k), \quad \Sigma \leftarrow \Sigma_Y(1:k, 1:k), \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(:, 1:k). \quad (5.3)$$

正如我们在第 4 节中讨论的, 我们可以通过仅更新小矩阵 Q_Y 并分别存储 Q_Y 和 $[Q \mid \tilde{e}]$ 来避免 (5.2) 中的时间复杂度 $\mathcal{O}(mr^2)$ 。不幸的是, 为了截断非常小的奇异值, 似乎很难在 $\mathcal{O}(r^2)$ 时间内找到两个正交矩阵 $\tilde{Q} \in \mathbb{R}^{m \times k}$ 和 $\tilde{Q} \in \mathbb{R}^{k \times k}$ 使得

$$[Q \mid \tilde{e}]Q_Y(:, 1:k) = \tilde{Q}\tilde{Q}.$$

因此, 我们直接使用 (5.2) 和 (5.3) 来更新矩阵 Q 和 R 。假设数据有 d 个非零奇异值, 那么这种新的截断需要 $\mathcal{O}((m+n)r^2d)$ 时间, 这里 r 是大于容差的奇异值的数量。

算法 10 (增量SVD (IV))

```

输入:  $Q \in \mathbb{R}^{m \times k}$ ,  $\Sigma \in \mathbb{R}^{k \times k}$ ,  $R \in \mathbb{R}^{k \times k}$ ,  $u_{+1} \in \mathbb{R}^m$ ,  $W \in \mathbb{R}^{m \times m}$ ,  $\text{tol}$ ,  $V$ ,  $Q_0$ ,  $q$ 
1: 设  $d = Q^T (W u_{+1})$ ;  $e = u_{+1} - Q d$ ;  $p = (e^T W e)^{1/2}$ ;
2: if  $p < \text{tol}$  then
3:    $q = q + 1$ ;
4:   设  $V\{q\} = d$ ;
5: else
6:   if  $q > 0$  then
7:     设  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
8:      $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, 'econ')$ ;
9:     设  $Q_0 = Q_0 Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,
10:     $R_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = [R * R_1; R_2]$ ;
11:    设  $d = Q_Y^T d$ 
12:   end if
13:   设  $e = e / p$ ;
14:   if  $|e^T W Q(:, 1)| > \text{tol}$  then
15:      $e = e - Q (Q^T (W e))$ ;  $p_1 = (e^T W e)^{1/2}$ ;  $e = e / p_1$ ;
16:   end if
17:   设  $Y = [\Sigma, d; 0, p]$ ;

```

```

18:    [Q_Y,  $\Sigma_Y$ , R_Y] = svd(Y);
19:    if  $\Sigma_Y(k+1, k+1) > \text{tol}$  then % 注意此处应为  $\Sigma_Y$ 
20:        设  $Q_0 = [Q_0, 0; 0, 1]$   $Q_Y$ ,  $Q = [Q, e]$   $Q_0$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,
21:         $R_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = [R * R_1; R_2]$ ,  $Q_0 = I_{\{k+1\}}$ ;
22:    else
23:        设  $Q_0 = [Q_0, 0; 0, 1]$   $Q_Y$ ,  $Q = [Q, e]$   $Q_0(:, 1:k)$ ,
24:         $\Sigma = \Sigma_Y(1:k, 1:k)$ ,  $R = [R, 0; 0, 1]$   $R_Y(:, 1:k)$ ,  $Q_0 = I_k$ 
25:    end if
26:     $V = []$ ;  $q = 0$ ;
27: end if
28: return Q,  $\Sigma$ , R, V,  $Q_0$ , q

```

接下来，我们在算法 11 和 12 中完成完整的实现。

算法 11 (增量SVD (IV) 最终检查)

输入: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{k \times k}$, V , Q_0 , q

```

1: if  $q > 0$  then
2:     设  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
3:     [Q_Y,  $\Sigma_Y$ , R_Y] = svd(Y, 'econ');
4:     设  $Q = Q Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,
5:      $R_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = [R * R_1; R_2]$ ;
6: end if
7: return Q,  $\Sigma$ , R.

```

算法 12 (完全增量SVD (IV))

输入: $W \in \mathbb{R}^{m \times m}$, tol

```

1: 获取  $u_1$ .
2: [Q,  $\Sigma$ , R] = InitializeISVD( $u_1$ , W) % 算法 1
3: 设  $V = []$ ;  $Q_0 = 1$ ;  $q = 0$ .
4: for  $i = 2, \dots, n$  do
5:     获取  $u_i$ 
6:     [q, V,  $Q_0$ , Q,  $\Sigma$ , R] = UpdateISVD4(q, V,  $Q_0$ , Q,  $\Sigma$ , R,  $u_i$ , W, tol) % 算法 10
7: end for
8: [Q,  $\Sigma$ , R] = UpdateISVD4check(q, V,  $Q_0$ , Q,  $\Sigma$ , R) % 算法 11
9: return Q,  $\Sigma$ , R

```

6 结论

在本文中，我们回答了 Brand 在 [4] 中提出的开放问题。我们证明了新算法的输出与 Brand 原始算法的输出相同。数值实验表明，我们的新算法不仅节省了 CPU，而且为左奇异向量矩阵保持了非常好的正交性。未来，我们将把我们的新算法与 [1] 中的许多其他增量 SVD 算法进行比较。除此之外，我们将把这个新算法应用于许多不同的问题，例如模型降阶和分数阶 PDE。

致谢

作者感谢王立涵博士对引理 3 的证明。

参考文献

- [1] C. BACH, D. CEGLIA, L. SONG, AND F. DUDDECK, Randomized low-rank approximation methods for projection-based model order reduction of large nonlinear dynamical problems, *Internat. J. Numer. Methods Engrg.*, 118 (2019), pp. 209-241.
- [2] M. W. BERRY, Large-scale sparse singular value computations, *The International Journal of Supercomputing Applications*, 6 (1992), pp. 13-49.
- [3] M. BRAND, Incremental singular value decomposition of uncertain data with missing values, in *European Conference on Computer Vision*, Springer, 2002, pp. 707-720.
- [4] M. BRAND, Fast low-rank modifications of the thin singular value decomposition, *Linear Algebra Appl.*, 415 (2006), pp. 20-30.
- [5] H. FAREED AND J. R. SINGLER, A note on incremental POD algorithms for continuous time data, *Appl. Numer. Math.*, 144 (2019), pp. 223-233.

- [6] H. FAREED AND J. R. SINGLER, Error analysis of an incremental proper orthogonal decomposition algorithm for PDE simulation data, *J. Comput. Appl. Math.*, 368 (2020), pp. 1125-1135.
- [7] H. FAREED, J. R. SINGLER, Y. ZHANG, AND J. SHEN, Incremental proper orthogonal decomposition for PDE simulation data, *Comput. Math. Appl.*, 75 (2018), pp. 1942-1960.
- [8] M. GHASHAMI, E. LIBERTY, J. M. PHILLIPS, AND D. P. WOODRUFF, Frequent directions: simple and deterministic matrix sketching, *SIAM J. Comput.*, 45 (2016), pp. 1762-1792.
- [9] S.-G. HWANG, Cauchy's interlace theorem for eigenvalues of Hermitian matrices, *Amer. Math. Monthly*, 111 (2004), pp. 157-159.
- [10] M. A. IWEN AND B. W. ONG, A distributed and incremental SVD algorithm for agglomerative data analysis on large networks, *SIAM J. Matrix Anal. Appl.*, 37 (2016), pp. 1699-1718.
- [11] L. LIN AND Y. TONG, Low-rank representation of tensor network operators with long-range pairwise interactions, *SIAM J. Sci. Comput.*, 43 (2021), pp. A164-A192.
- [12] N. MASTRONARDI, E. E. TYRTYSHNIKOV, AND P. VAN DOOREN, A fast algorithm for updating and downsizing the dominant kernel principal components, *SIAM J. Matrix Anal. Appl.*, 31 (2010), pp. 2376-2399.
- [13] G. M. OXBERRY, T. KOSTOVA-VASSILEVSKA, W. ARRIGHI, AND K. CHAND, Limited-memory adaptive snapshot selection for proper orthogonal decomposition, *Internat. J. Numer. Methods Eng.*, 109 (2017), pp. 198-217.

四、外文原文

An answer to an open question in the incremental SVD

Yangwen Zhang*

May 3, 2022

Abstract

Incremental singular value decomposition (SVD) was proposed by Brand to efficiently compute the SVD of a matrix. The algorithm needs to compute thousands or millions of orthogonal matrices and to multiply them together. However, the multiplications may corrode the orthogonality. Hence many reorthogonalizations are needed in practice. In [Linear Algebra and its Applications 415 (2006) 20–30], Brand asked “It is an open question how often this is necessary to guarantee a certain overall level of numerical precision; it does not change the overall complexity.” In this paper, we answer this question and the answer is we can avoid computing the large amount of those orthogonal matrices and hence the reorthogonalizations are not necessary by modifying his algorithm. We prove that the modification does not change the outcomes of the algorithm. Numerical experiments are presented to illustrate the performance of our modification.

1 Introduction

The singular value decomposition (SVD) of a matrix has many applications, such as proper orthogonal decomposition model order reduction and principal component analysis.

One drawback of the SVD is the cost of its computation. Let U be a m by n dense matrix of low rank r , the computational complexity of traditional methods is $O(mn^2 + m^2n + n^3)$ time, which is unfeasible for a large size matrix. Lanczos methods [2] yield thin svds and the complexity is $O(mnr^2)$ time, but we need to know the rank in advance. These methods are referred to as *batch* methods because they need to store the matrix.

In some scenarios the data sets will be produced incrementally, such as the snapshots of a time dependent partial differential equations (PDEs). It may be advantageous to perform the SVD as the columns of a matrix become available, instead of waiting until all data sets are available before doing any computation. These characteristics on the availability of data sets have given rise to a class of incremental methods.

In 2002, Brand [3] proposed a new algorithm to find the SVD of a matrix incrementally. Given the SVD of a matrix $U = Q\Sigma R^\top$, the goal is to update the SVD of the related matrix $[U \mid c]$, by using the SVD of U and the new adding vector c . Then the SVD of $[U \mid c]$ can be constructed by

1. letting $e = c - QQ^\top c$ and $p = \|e\|$,
2. finding the full SVD of $\begin{bmatrix} \Sigma & Q^\top c \\ 0 & p \end{bmatrix} = \tilde{Q}\tilde{\Sigma}\tilde{R}^\top$, and then

*Department of Mathematics Science, Carnegie Mellon University, Pittsburgh, PA, USA (yangwenz@andrew.cmu.edu).

3. updating the SVD of $[U \mid c]$ by

$$[U \mid c] = ([Q \mid e/p] \tilde{Q}) \tilde{\Sigma} \left(\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} \tilde{R} \right)^\top.$$

For many of the motivating applications, only the dominant singular vectors and values of U are needed. Hence, in practice, we perform truncation when p or the last singular value of $\tilde{\Sigma}$ is small. The incremental SVD has been applied to many different areas; see [8, 10–12] for more details. There are many other incremental methods, see [1] for a survey.

There are two ways to update the left and right singular subspace. The first way is very straightforward and the update is given by

$$Q \leftarrow [Q \mid e/p] \tilde{Q}, \quad \Sigma \leftarrow \tilde{\Sigma}, \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} \tilde{R}. \quad (1.1)$$

However, this update has two defects:

- (1) The computational cost of (1.1) is expensive since the updates involves a very tall thin matrix Q and R .
- (2) Reorthogonalizations are needed for Q almost after each update. Theoretically, the matrix $[Q \mid e/p] \tilde{Q}$ is orthogonal. However, in practice, small numerical errors cause a loss of orthogonality. Without a reorthogonalization, the decomposition of the incremental SVD algorithm is not orthogonal. Therefore, the singular values are not true and truncation is not reliable. Hence a reorthogonalization is needed and the computational cost can be high if the rank of the matrix is not extremely low. Specifically, Fareed et al. recommended a Gram-Schmidt orthogonalization [7] while Oxberry et al. chose thin QR in [13].

Numerical experiments in Example 1 show that the above two items take the majority of the CPU time.

To avoid the huge computational cost, Brand [4] suggested to update the matrix Q and R indirectly. His idea is to leave the SVD decomposed into the five matrices

$$U = Q_{\text{old}} \tilde{Q}_{\text{old}} \Sigma_{\text{old}} \tilde{R}_{\text{old}}^\top R_{\text{old}}^\top$$

with orthonormal $Q_{\text{old}} \tilde{Q}_{\text{old}}$, $R_{\text{old}} \tilde{R}_{\text{old}}$, Q_{old} and $\tilde{Q}_{\text{old}}$ (but not $\tilde{R}_{\text{old}}$ or $R_{\text{old}}^\top$). The large outer matrices only record the span of the left and right subspaces and are built by appending columns to Q_{old} and rows to R_{old} . The updates only involve with the small matrices $\tilde{Q}_{\text{old}}$ and $\tilde{R}_{\text{old}}$. This makes the update much faster.

However, over thousands or millions of updates, the multiplications may erode the orthogonality of through numerical error; see the right figure in Figure 3. Hence reorthogonalizations are needed for these small orthonormal matrices. In [4], Brand asked: “It is an open question how often this is necessary to guarantee a certain overall level of numerical precision; it does not change the overall complexity.” We also note that only reorthogonalize the small matrix $\tilde{Q}$ is not enough for some data sets. In Example 2, we show that without reorthogonalizations for the larger outside matrix Q_{old} , the algorithm provides untruthful SVD.

In this paper, we first answer the open question, and the answer is we can avoid computing the large amount of those orthogonal matrices and hence the reorthogonalizations for those small orthonormal matrices are not necessary by modifying Brand’s algorithm. Second, we proposed an efficient reorthogonalization for the larger out matrix Q_{old} , numerical experiments in Examples 2 and 3 show that the reorthogonalization works well in practice. Third, we prove that our modification does not change the outputs of Brand’s algorithm.

2 Incremental SVD (I)

We begin by introducing notation needed throughout the paper. Let I_k be a $k \times k$ identity matrix and the W -weighted inner product:

$$(a, b)_W = a^\top W b, \quad a, b \in \mathbb{R}^m.$$

Let U be a m by n dense matrix and u_j denote the j -th column of U , i.e.,

$$U = [u_1 \mid u_2 \mid \cdots \mid u_n].$$

For convenience, we adopt Matlab notation herein. We use both $U(:, 1 : \ell)$ and U_ℓ to denote the first ℓ columns of U , and $U(1 : \ell, 1 : \ell)$ be the ℓ -th leading principal minor of U . The function **diag**: $\mathbb{R}^k \rightarrow \mathbb{R}^{k \times k}$ takes as input a vector and outputs a square, diagonal matrix with that vector's entries on its main diagonal as follows: if $\alpha = (a_1, a_2, \dots, a_k)^\top$, then

$$\text{diag}(\alpha) = \begin{bmatrix} a_1 & & & \\ & a_2 & & \\ & & \ddots & \\ & & & a_k \end{bmatrix}.$$

If the data arising from a Galerkin-type simulation of a PDE, one has to deal with Cholesky factorization of a weighted matrix if one directly apply Brand's algorithm. To avoid this, Fareed et al. [7] extended Brand's algorithm to accommodate data of this type without computing Cholesky factorization. More specifically, if the data lies in a finite element space and is expressed using a collection of basis functions. Then the SVD of the data is equivalent to find a so-called core SVD of their coefficient matrix.

Definition 1. [7] A core SVD of a matrix $U \in \mathbb{R}^{m \times n}$ is a decomposition $U = Q\Sigma R^\top$, where $Q \in \mathbb{R}^{m \times d}$, $\Sigma \in \mathbb{R}^{d \times d}$, and $R \in \mathbb{R}^{n \times d}$ satisfy

$$Q^\top W Q = I, \quad R^\top R = I, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_d),$$

where $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_d > 0$. The values $\{\sigma_i\}$ are called the (positive) singular values of U and the columns of Q and R are called the corresponding singular vectors of U .

We note that if $W = I$, the above core SVD is reduced to the standard SVD of the matrix U . In [7], W is the standard finite element mass or stiffness matrix. For more information about the incremental SVD in a weighted norm setting, see [5, 6].

Next, we introduce Brand's incremental SVD algorithm and follow the discussions in [7].

Step 1: Initialization. Assume that the first column of U is nonzero, i.e., $u_1 \neq 0$, we initialize the core SVD of u_1 by setting

$$\Sigma = (u_1^\top W u_1)^{1/2}, \quad Q = u_1 \Sigma^{-1}, \quad R = 1.$$

The algorithm is shown in Algorithm 1.

Step 2: Core SVD of U_ℓ . Suppose we already have a rank- k truncated core SVD of U_ℓ :

$$U_\ell = Q\Sigma R^\top, \quad \text{with } Q^\top W Q = I_k, \quad R^\top R = I_k, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_k), \quad (2.1)$$

where $\Sigma \in \mathbb{R}^{k \times k}$ is a diagonal matrix with the k (ordered) singular values of U_ℓ on the diagonal, $Q \in \mathbb{R}^{m \times k}$ is the matrix containing the corresponding k left singular vectors of U_ℓ , and $R \in \mathbb{R}^{\ell \times k}$ is the matrix of the corresponding k right singular vectors of U_ℓ .

Algorithm 1 (Initialize incremental SVD)

Input: $u_1 \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$

- 1: $\Sigma = (u_1^\top W u_1)^{1/2}$; $Q = u_1 \Sigma^{-1}$; $R = 1$;
 - 2: **return** Q, Σ, R
-

Step 3: Updates the core SVD of $U_{\ell+1}$. Our goal next is to update the above SVD, i.e., we want to find the core SVD of $U_{\ell+1}$ by using the core SVD of U_ℓ and the new adding vector $u_{\ell+1}$.

To do this, let $e \in \mathbb{R}^m$ be the residual of $u_{\ell+1}$ onto the space spanned by the columns of Q with the weighted inner product $(\cdot, \cdot)_W$. Therefore,

$$e = u_{\ell+1} - QQ^\top W u_{\ell+1}.$$

Let p be the magnitude of e , i.e., $p = (e^\top W e)^{1/2}$. If $p > 0$, we let $\tilde{e}$ be the unit vector in the direction of e , i.e., $\tilde{e} = e/p$. If $p = 0$, we set $\tilde{e} = 0$. Then we have the fundamental identity:

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = [Q \Sigma R^\top \mid u_{\ell+1}] = [Q \mid \tilde{e}] \underbrace{\begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_Y \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (2.2)$$

We can find the SVD of the updated matrix $U_{\ell+1}$ by finding the full SVD of the matrix Y in the right hand side of the above identity.

- (1) If $p < \text{tol}$, we approximate and set $p = 0$ and $\tilde{e} = 0$. Let $Q_Y \Sigma_Y R_Y^\top$ be the full SVD of $Y = \begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & 0 \end{bmatrix}$. Then the core SVD of $U_{\ell+1} \in \mathbb{R}^{m \times (\ell+1)}$ is given by

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = ([Q \mid 0] Q_Y) \Sigma_Y \left(\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y \right)^\top.$$

This suggests the following update

$$Q \leftarrow QQ_Y(1:k, 1:k), \quad \Sigma \leftarrow \Sigma_Y(1:k, 1:k), \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(:, 1:k). \quad (2.3)$$

- (2) If $p \geq \text{tol}$, we let $Q_Y \Sigma_Y R_Y^\top$ be the full SVD of the middle matrix Y in (2.2), and then update the SVD of $U_{\ell+1}$ by

$$Q \leftarrow [Q \mid \tilde{e}] Q_Y \quad \Sigma \leftarrow \Sigma_Y \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y. \quad (2.4)$$

In [7], the authors considered a data from the simulation of 2D laminar flow around a cylinder with circular cross-section. The incremental SVD algorithms kept computing very small singular values (near machine precision) - these required huge computational cost and these values were not needed in their application. Hence, they suggested another truncation in [7, Algorithm 4] if the last few singular values are less than a tolerance. That is, find the minimal r such that $\Sigma_Y(r, r) \geq \text{tol}$ and $\Sigma_Y(r+1, r+1) < \text{tol}$ and set the following truncation:

$$Q \leftarrow Q(:, 1:r), \quad \Sigma \leftarrow \Sigma(1:r, 1:r), \quad R \leftarrow R(:, 1:r), \quad (2.5)$$

Algorithm 2 (Update Incremental SVD (I))

Input: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{\ell \times k}$, $u_{\ell+1} \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$, **tol**

```

1: Set  $d = Q^\top (W u_{\ell+1})$ ;  $e = u_{\ell+1} - Qd$ ;  $p = (e^\top W e)^{1/2}$ ;
2: if  $p < \text{tol}$  then
3:   Set  $p = 0$ ;
4: else
5:   Set  $e = e/p$ ;
6: end if
7: Set  $Y = \begin{bmatrix} \Sigma & d \\ 0 & p \end{bmatrix}$ ;
8:  $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$ ;
9: if  $p < \text{tol}$  then
10:   Set  $Q = Q Q_Y(1:k, 1:k)$ ,  $\Sigma = \Sigma_Y(1:k, 1:k)$ ,  $R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(:, 1:k)$ ;
11: else
12:   Set  $Q = [Q \mid e] Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y$ .
13:   if  $\Sigma(r, r) \geq \text{tol} \& \Sigma(r+1, r+1) \geq \text{tol}$  then
14:     Set  $Q = Q(:, 1:r)$ ,  $\Sigma = \Sigma(1:r, 1:r)$ ,  $R = R(:, 1:r)$ ;
15:   end if
16: end if
17: return  $Q, \Sigma, R$ 

```

where Q, Σ and R were defined in (2.4). Now we summarize the above discussions in Algorithm 2.

Next, we discuss the complexity of Brand's algorithm (Algorithm 2) with $W = I$ and note that $U \in \mathbb{R}^{m \times n}$ with low rank r . The total time complexity is $\mathcal{O}((m+n)r^2)$ and the details are shown below.

1. Line 1: this needs to be executed n times and the computational cost of each time is about $\mathcal{O}(mr)$, the whole complexity is $\mathcal{O}(mnr)$.
2. Lines 8: since Y is a one-column bordered diagonal matrix, then it can be bidiagonalized in $\mathcal{O}(r^2)$ time and the whole time is $\mathcal{O}(nr^2)$.
3. Lines 10-12: the computational cost of each time is about $\mathcal{O}(mr^2 + \ell r^2)$, hence the total complexity is about $\mathcal{O}(\sum_{\ell=1}^n (mr^2 + \ell r^2)) = \mathcal{O}((m+n)nr^2)$.

Step 4: Reorthogonalization. Theoretically, the above SVD update yields orthonormal left and right singular vectors. However, in practice, small numerical errors cause a loss of orthogonality. Then the incremental SVD algorithm is a non-orthogonal decomposition, and the singular values are not true hence truncation is not reliable. Therefore a reorthogonalization is needed and the computational cost can be high if the rank of the matrix is not extremely low. Specifically, Brand recommended a Gram-Schmidt orthogonalization [3] while Oxberry et al. chose thin QR in [13]. Our numerical test show that thin QR is faster than the Gram-Schmidt when the weighted matrix is an identity, see Example 1. However, since there is no thin QR with a weighted norm setting at hand, Fareed et al. in [7] use the W -weighted Gram-Schmidt for the reorthogonalization.

Algorithm 3 (Reorthogonalization)

Input: $Q \in \mathbb{R}^{m \times k}$, $W \in \mathbb{R}^{m \times m}$, tol

```

1: if  $|(Q(:, \text{end}))^\top W Q(:, 1)| > \text{tol}$  then
2:   for  $i = 1$  to  $k$  do
3:     Set  $\alpha = Q(:, i)$ ;
4:     for  $j = 1$  to  $i - 1$  do
5:        $Q(:, i) = Q(:, i) - (\alpha^\top W Q(:, j))Q(:, j)$ ;
6:     end for
7:     Set  $\text{norm} = ((Q(:, i))^\top W Q(:, i))^{1/2}$ ;
8:     Set  $Q(:, i) = Q(:, i) / \text{norm}$ ;
9:   end for
10: end if
11: return  $Q$ 

```

We do not discuss the time complexity of the reorthogonalization since there is no clue that how many times we need to execute the Algorithm 3. However, numerical experiments in Example 1 show that the orthogonalization step described above is a large part of the computational cost of the incremental SVD algorithm.

Finally we conclude the full implementation of the incremental SVD with the W -weighted Gram–Schmidt for the reorthogonalization in Algorithm 4.

Algorithm 4 (Fully incremental SVD (I))

Input: $W \in \mathbb{R}^{m \times m}$, tol

```

1: Get  $u_1$ ;
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$ ; % Algorithm 1
3: for  $\ell = 2, \dots, n$  do
4:   Get  $u_\ell$ ;
5:    $[Q, \Sigma, R] = \text{UpdateISVD1}(Q, \Sigma, R, u_\ell, W, \text{tol})$ ; % Algorithm 2
6:    $Q = \text{Reorthogonalization}(Q, W, \text{tol})$ ; % Algorithm 3
7: end for
8: return  $Q, \Sigma, R$ 

```

Example 1. In this example, we test the Algorithm 4. We let $\Omega \subset \mathbb{R}^2$ be an unit square and we partition it into 524288 uniform triangles. Let $\{\varphi_i\}_{i=1}^m$ be the standard linear finite element basis functions. Next we let $\{(x_i, y_j)\}_{i,j=1}^m$ be the grids of Ω , $\{t_i\}_{i=1}^n$ be an equally space grid in $[0, 10]$ and time step $\Delta t = 1/10^3$. Define

$$f(t, x, y) = \cos(t(x + y)), \quad b_i = [(f(t_i), \varphi_j)]_{j=1}^m, \quad B = [b_1 \mid b_2 \mid \dots \mid b_n],$$

$$M = [(\varphi_j, \varphi_i)]_{i,j=1}^m, \quad u_k = [f(t_k, x_i, y_j)]_{i,j=1}^m, \quad U = [u_1 \mid u_2 \mid \dots \mid u_n].$$

Here M is the finite element mass matrix and u_k is the coefficient of the linear Lagrange interpolation of $f(t, x, y)$ at $t = t_k$. We compute the SVD of the matrix B with $W = I$ and of the matrix U with $W = M$ by setting $\text{tol} = 10^{-12}$. Let $\|A\|_F$ be the Frobenius norm of matrix A , and we use

$$\mathcal{E}_W = \|I - Q^\top W Q\|$$

to measure the error of the orthogonality of the orthonormal matrix Q under the weight inner product $(\cdot, \cdot)_W$.

In Figure 1 we show the orthogonality of the left singular matrix Q with different weighted norms. For the case $W = M$, the orthogonality of Q is eroded (even with reorthogonalizations).

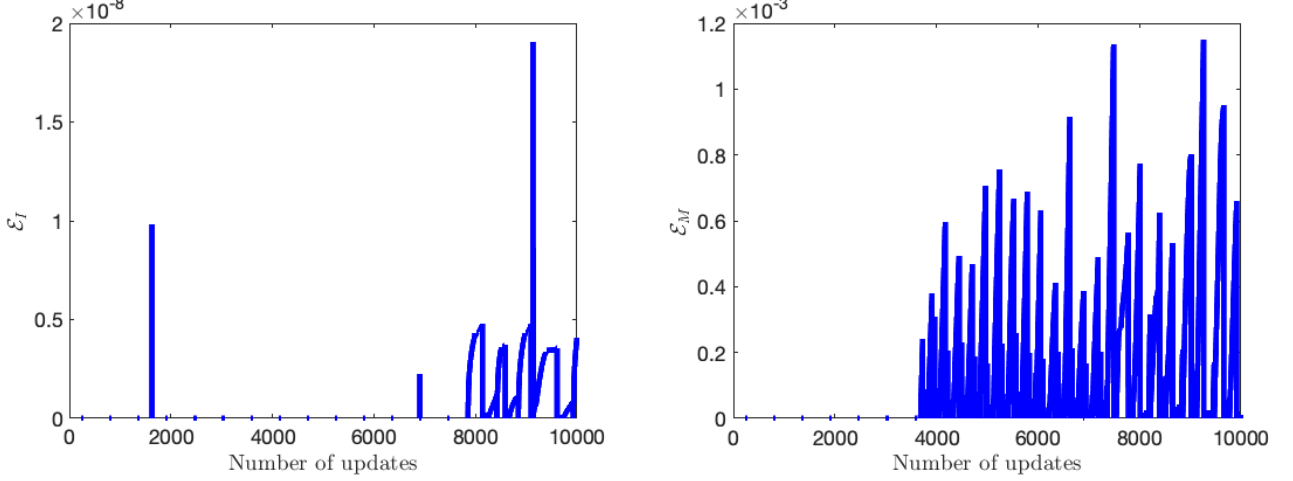


Figure 1: Example 1: Left: $W = I$. Right: $W = M$.

The whole simulations take about 154 and 4000 seconds for $W = I$ and $W = M$, respectively. Next, we show the CPU¹ time (seconds) of the main part of the Algorithm 4 in Table 1. We see that the orthogonalization step is a large part of the computational cost of the incremental SVD algorithm, especially for the case $W = M$.

	Line 1 in Algorithm 2	Lines 9-13 in Algorithm 2	Algorithm 3
$W = I$	32.034	60.213	60.129
$W = M$	110.58	115.96	3761.6

Table 1: The CPU time (seconds) for different weighted inner products.

3 Incremental SVD (II)

Algorithm 4 is easy to implement while the computational cost is high, especially for the weighted norm setting. For a $m \times n$ dense matrix with low rank r , the time complexity of Algorithm 4 is $\mathcal{O}(m(m+n)r^2)$. In [4], Brand reduced the complexity to $\mathcal{O}(mnr)$. However, to update the left singular space, one has to compute thousands or millions of orthogonal matrices and multiply them together. It is more complicate to update the right singular space which need to compute the pseudo-inverse of a matrix for thousands or millions times.

Next, we follow [4] to extend Brand's improvements to compute the core SVD of $U \in \mathbb{R}^{m \times n}$ with respect to the weighted inner product $(\cdot, \cdot)_W$. Instead of updating the large singular vector matrices as prescribed in (2.3) or (2.4), we suppose the core SVD of $U_\ell \in \mathbb{R}^{m \times \ell}$ can be written into the following five matrices

$$U_\ell = Q_{\text{old}} \tilde{Q}_{\text{old}} \Sigma_{\text{old}} \tilde{R}_{\text{old}}^\top R_{\text{old}}^\top. \quad (3.1)$$

¹All the code for all examples in the paper has been made by the author using MATLAB R2020b and has been run on a laptop with MacBook Pro, 2.3 Ghz8-Core Intel Core i9 with 64GB 2667 Mhz DDR4.

Here $Q_{\text{old}} \in \mathbb{R}^{m \times k}$ is a orthonormal matrix under the weighted inner product, $\tilde{Q}_{\text{old}}$ and $\tilde{R}_{\text{old}}^\top R_{\text{old}}^\top$ are standard orthonormal matrices, but $\tilde{R}_{\text{old}}^\top$ and $R_{\text{old}}^\top$ are not. The large outer matrices only record the span of the left and right subspaces and are built by appending columns to Q_{old} and rows to $R_{\text{old}}^\top$. And the update only relates to the small matrices $\tilde{Q}_{\text{old}}$ and $\tilde{R}_{\text{old}}^\top$, this makes the update much faster.

Next, we consider the core SVD of $U_{\ell+1} = [U_\ell \mid u_{\ell+1}] \in \mathbb{R}^{m \times (\ell+1)}$. Similar to Section 2, we let $e \in \mathbb{R}^m$ be the residual of $u_{\ell+1}$ onto the space spanned by the columns of $Q_{\text{old}}\tilde{Q}_{\text{old}}$ with the weighted inner product $(\cdot, \cdot)_W$, i.e.,

$$e = u_{\ell+1} - (Q_{\text{old}}\tilde{Q}_{\text{old}})(Q_{\text{old}}\tilde{Q}_{\text{old}})^\top W u_{\ell+1} = u_{\ell+1} - Q_{\text{old}}Q_{\text{old}}^\top W u_{\ell+1}.$$

Let $p = (e^\top W e)^{1/2}$. If $p > 0$, we let $\tilde{e} = e/p$, otherwise we set $\tilde{e} = 0$. Then by (3.1) and the fundamental identity (2.2) we have

$$U_{\ell+1} = \left[Q_{\text{old}}\tilde{Q}_{\text{old}} \mid \tilde{e} \right] \underbrace{\begin{bmatrix} \Sigma_{\text{old}} & \tilde{Q}_{\text{old}}Q_{\text{old}}^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_Y \begin{bmatrix} R_{\text{old}}\tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix}^\top.$$

Assume that $Q_Y \Sigma_Y R_Y^\top$ be the full SVD of Y , then the core SVD of $U_{\ell+1}$ is given by

$$U_{\ell+1} = \left(\left[Q_{\text{old}}\tilde{Q}_{\text{old}} \mid \tilde{e} \right] Q_Y \right) \Sigma_Y \left(\begin{bmatrix} R_{\text{old}}\tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} R_Y \right)^\top. \quad (3.2)$$

- (1) If $p < \text{tol}$, we set $p = 0$ and $\tilde{e} = 0$, then the update is not rank-increasing. We update the small orthonormal matrix $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times k}$ and the diagonal matrix $\Sigma_{\text{new}} \in \mathbb{R}^{k \times k}$ by

$$\tilde{Q}_{\text{new}} \leftarrow \tilde{Q}_{\text{old}} Q_Y(1:k, 1:k), \quad \Sigma_{\text{new}} \leftarrow \Sigma_Y(1:k, 1:k). \quad (3.3)$$

- (2) If $p \geq \text{tol}$, we then update the large outer matrix Q_{new} by adding the vector $\tilde{e}$ to Q_{old} , and update the small orthonormal matrix $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times (k+1)}$ and the diagonal matrix $\Sigma_{\text{new}} \in \mathbb{R}^{(k+1) \times (k+1)}$ by

$$Q_{\text{new}} \leftarrow [Q_{\text{old}} \mid \tilde{e}], \quad \tilde{Q}_{\text{new}} \leftarrow \begin{bmatrix} \tilde{Q}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} Q_Y, \quad \Sigma_{\text{new}} \leftarrow \Sigma_Y. \quad (3.4)$$

The time complexity of (3.3) or (3.4) is about $\mathcal{O}(r^2)$ since the update of the left singular vector matrix is completed by the multiplication of two small orthonormal matrices. This is a major difference comparing to (2.3) or (2.4).

However, we have to update the small orthonormal matrices over thousands or millions of times, the multiplications may erode the orthogonality of $\tilde{Q}$ through numerical error. Hence reorthogonalizations are needed when the inner product between the first and last left singular vectors of $\tilde{Q}$ is greater than some tolerance.

In [4], Brand asked ‘‘It is an open question how often this is necessary to guarantee a certain overall level of numerical precision; it does not change the overall complexity’’. In the next section, we answer this question and the answer is we can avoid computing the large amount of those orthogonal matrices and those multiplications are not necessary. Hence, the reorthogonalizations are not needed.

Remark 1. Theoretically, the new adding vector $\tilde{e}$ is orthogonal to each column of Q_{old} . However, in practice, the matrix $[Q_{\text{old}} \mid \tilde{e}]$ loss its orthogonality very quickly for some data sets. Hence reorthogonalizations are needed for this matrix. We note that in [4], Brand did not include this. In Example 2, we show how this is important and in the next section we propose a very efficient way to remedy this issue.

The updates of right singular vector matrix R_{new} and $\tilde{R}_{\text{new}}$ are more complicated because of adding rows to R_{old} while guaranteeing that the columns of the product $R_{\text{new}}\tilde{R}_{\text{new}}$ are orthonormal. From the fundamental identity (2.2) and (3.2), the right-side update must satisfy

$$R_{\text{new}}\tilde{R}_{\text{new}} = \begin{bmatrix} R_{\text{old}}\tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} R_Y.$$

In the following updates, we need to compute the pseudo-inverse of $\tilde{R}_{\text{old}}$ and a sub-matrix of R_Y . We use X^+ to denote the pseudo-inverse of X .

- (3) If $p < \text{tol}$, the rank does not increase, the last column of R_Y represents an unused subspace dimension and should be suppressed. Let $\mathcal{R}_1 = R_Y(1:k, 1:k)$, $\mathcal{R}_2 = R_Y(k+1, 1:k)$,

$$\tilde{R}_{\text{new}} \leftarrow \tilde{R}_{\text{old}}\mathcal{R}_1, \quad \tilde{R}_{\text{new}}^+ \leftarrow \mathcal{R}_1^+ \tilde{R}_{\text{old}}^+, \quad R_{\text{new}} \leftarrow \begin{bmatrix} R_{\text{old}} \\ \mathcal{R}_2 \tilde{R}_{\text{new}}^+ \end{bmatrix}. \quad (3.5)$$

- (4) If $p \geq \text{tol}$, the update is rank-increasing. We update the small orthonormal matrix $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times k}$ and the diagonal matrix $\Sigma_{\text{new}} \in \mathbb{R}^{k \times k}$ by

$$\tilde{R}_{\text{new}} \leftarrow \begin{bmatrix} \tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} R_Y, \quad \tilde{R}_{\text{new}}^+ \leftarrow R_Y^\top \begin{bmatrix} \tilde{R}_{\text{old}}^+ & 0 \\ 0 & 1 \end{bmatrix}, \quad R_{\text{new}} \leftarrow \begin{bmatrix} R_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix}. \quad (3.6)$$

The implementation of this incremental SVD algorithm is shown in Algorithms 5 and 6.

Algorithm 5 (Incremental SVD (II))

Input: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{\ell \times k}$, $u_{\ell+1} \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$, tol , $\tilde{Q}$, $\tilde{R}$, $\tilde{R}^+$

- 1: Set $d = Q^\top (W u_{\ell+1})$; $e = u_{\ell+1} - Qd$; $p = (e^\top W e)^{1/2}$;
- 2: **if** $p < \text{tol}$ **then**
- 3: Set $Y = \begin{bmatrix} \Sigma & d \\ 0 & 0 \end{bmatrix}$;
- 4: $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$;
- 5: Set $\tilde{Q} = \tilde{Q}Q_Y$, $\Sigma = \Sigma_Y$, $\mathcal{R}_1 = R_Y(1:k, 1:k)$, $\mathcal{R}_2 = R_Y(k+1, 1:k)$,
- 6: $\tilde{R} = \tilde{R}\mathcal{R}_1$, $\tilde{R}^+ = \mathcal{R}_1^+ \tilde{R}^+$, $R = \begin{bmatrix} R \\ \mathcal{R}_2 \tilde{R}^+ \end{bmatrix}$;
- 7: **else**
- 8: Set $e = e/p$;
- 9: Set $Y = \begin{bmatrix} \Sigma & d \\ 0 & p \end{bmatrix}$;
- 10: $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$;
- 11: Set $\tilde{Q} = \begin{bmatrix} \tilde{Q} & 0 \\ 0 & 1 \end{bmatrix} Q_Y$, $Q = [Q \mid e]$, $\Sigma = \Sigma_Y$, $R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}$,
- 12: $\tilde{R} = \begin{bmatrix} \tilde{R} & 0 \\ 0 & 1 \end{bmatrix} R_Y$, $\tilde{R}^+ = R_Y^\top \begin{bmatrix} \tilde{R}^+ & 0 \\ 0 & 1 \end{bmatrix}$;
- 13: **end if**
- 14: **if** $|(\tilde{Q}(:, 1))^\top \cdot \tilde{Q}(:, \text{end})| > \text{tol}$ **then**
- 15: Apply Gram-Schmidt to Reorthogonalize the matrix $\tilde{Q}$;
- 16: **end if**
- 17: **return** $Q, \Sigma, R, \tilde{Q}, \tilde{R}, \tilde{R}^+$

Algorithm 6 (Fully incremental SVD (II))

Input: $W \in \mathbb{R}^{m \times m}$, tol

- 1: Get u_1 ;
- 2: $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$; % Algorithm 1
- 3: Set $\tilde{Q} = \tilde{R} = 1$;
- 4: **for** $\ell = 2, \dots, n$ **do**
- 5: Get u_ℓ
- 6: $[Q, \Sigma, R, \tilde{Q}, \tilde{R}, \tilde{R}^+] = \text{UpdateISVD2}(Q, \Sigma, R, \tilde{Q}, \tilde{R}, \tilde{R}^+, u_\ell, W, \text{tol})$; % Algorithm 5
- 7: **end for**
- 8: **return** $Q, \Sigma, R, \tilde{Q}, \tilde{R}$

Example 2. In this example, we first show that the reorthogonalizations are needed for the matrix Q in Algorithm 5. Let $\Omega \subset \mathbb{R}^2$ be an unit square and partition it into 512 uniform triangles. Let $\{\varphi_i\}_{i=1}^m$ be the standard linear finite element basis functions, $\{t_i\}_{i=1}^n$ be an equally space grid in $[0, 10]$ and time step $\Delta t = 1/10^2$. Define

$$f(t, x, y) = \cos(t(x + y)), \quad b_i = [(f(t_i), \varphi_j)]_{j=1}^m, \quad B = [b_1 \mid b_2 \mid \dots \mid b_n].$$

We then use the Algorithm 6 to compute the SVD of B with the weighted matrix $W = I$. To make a comparison, we use the MATLAB built-in function `svd()` to find the “exact” singular values of B . From Figure 2 we see that Brand’s algorithm provides unrelated singular values.

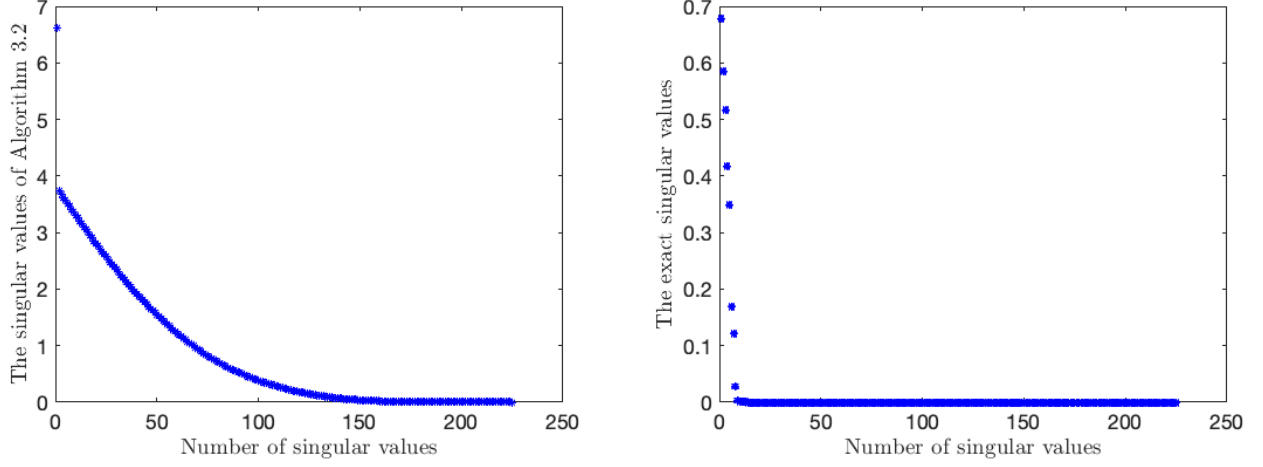


Figure 2: Singular values of B : Left: Algorithm 6. Right: MATLAB built-in function `svd()`.

Next, we add reorthogonalizations (details will be given in the next section) to Algorithm 6 and plot the errors (left in Figure 3) between the modified Algorithm 6 and the MATLAB built-in function `svd()`. We see that the new adding reorthogonalizations works well in practice.

Finally, we test the orthogonality of the small orthonormal matrices $\tilde{Q}$ at each update, we use $\mathcal{E}_I(\tilde{Q}) = \|I - \tilde{Q}^\top \tilde{Q}\|$ to measure the orthogonality of $\tilde{Q}$, which is shown in the right of Figure 3, we see that the matrix $\tilde{Q}$ loss its orthogonality quickly after $n = 600$ even performed the reorthogonalizations.

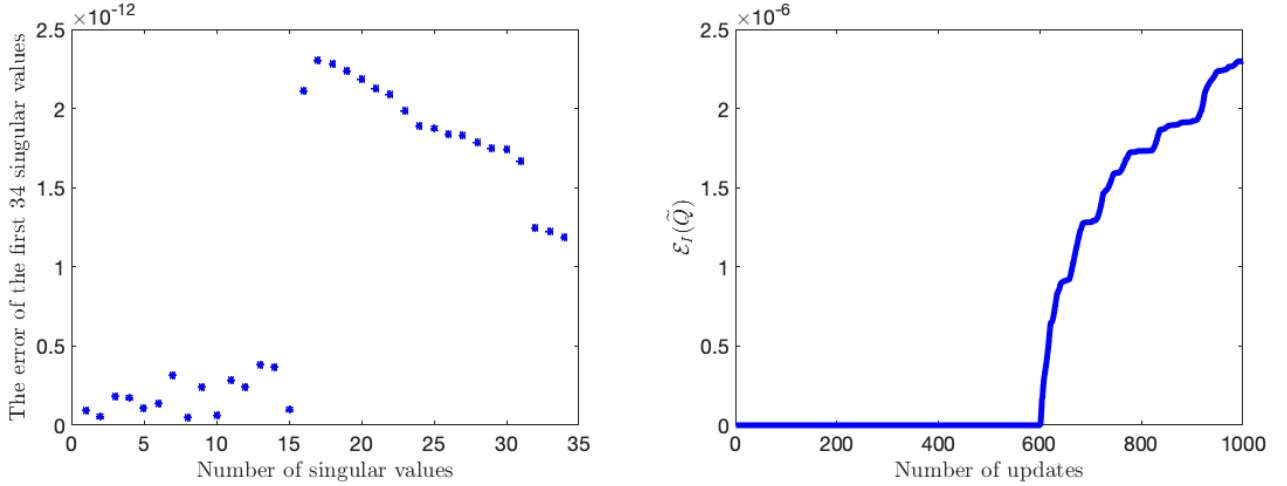


Figure 3: Example 2: Left: the errors of the first 34 singular values of B by adding the reorthogonalizations. Right: the error of the orthogonality of $\tilde{Q}$ at each update.

4 Our improvements

As it is pointed in Section 3, the Algorithm 6 needs to update the small orthonormal matrices $\tilde{Q}$ over thousands or millions of times, the multiplications may erode the orthogonality of $\tilde{Q}$ through

numerical error. To overcome this issue, Brand proposed reorthogonalizations when the inner product between the first and last left singular vectors of $\tilde{Q}$ is greater than some tolerance. However, as we have seen in Example 2, even we performed the reorthogonalizations, the orthonormal matrix $\tilde{Q}$ could still lose its orthogonality. Furthermore, to update the right singular space, the Algorithm 6 needs to compute the pseudo-inverse of a matrix for thousands or millions of times.

First, we summarize our contributions in this section.

- (1) We answer the question that Brand asked in [4]: “It is an open question how often this is necessary to guarantee a certain overall level of numerical precision; it does not change the overall complexity”.
- (2) We prove that the output of our new algorithm is the same with Brand’s.
- (3) We show that it is not necessary to compute thousands or millions of the pseudo-inverse of a matrix to update the right singular subspace.
- (4) We propose a reorthogonalization for the matrix $[Q_{\text{old}} \mid \tilde{e}]$, we note that this step is not in the original algorithm in [4]. In Example 2 we have showed that the reorthogonalizations are necessary.

To begin, we introduce the following lemma:

Lemma 1. Let $A \in \mathbb{R}^{k \times k}$, $\alpha \in \mathbb{R}^k$, and let $Q_1 \Sigma_1 R_1^\top$ be a thin SVD of $[A \mid \alpha]$, then

$$\begin{bmatrix} Q_1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} R_1^\top \\ r_{k+1}^\top \end{bmatrix} \quad (4.1)$$

is a SVD of $\begin{bmatrix} A & \alpha \\ 0 & 0 \end{bmatrix}$, where $r_{k+1} \in \mathbb{R}^{k+1}$ is a unit vector and orthogonal to each column of R_1 .

Next, we use Lemma 1 to modify the updates of the left singular subspace if the rank does not increase. Assume that $Q \Sigma R^\top$ is a rank k truncated core SVD of U_ℓ , the norm of $u_{\ell+1} - Q Q^\top W u_{\ell+1}$ is very small and we approximate it by 0. Therefore,

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = Q \begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \end{bmatrix} \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (4.2)$$

Let $Q_1 \Sigma_1 R_1^\top$ be a thin SVD of $[\Sigma \mid Q^\top W u_{\ell+1}]$, then the core SVD of $U_{\ell+1}$ is given by

$$U_{\ell+1} = Q Q_1 \Sigma_1 R_1^\top \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (4.3)$$

Hence the following update

$$Q \leftarrow Q Q_1, \quad \Sigma = \Sigma_1, \quad R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_1, \quad (4.4)$$

is exactly the same with (2.3) due to Lemma 1.

Next, we consider a more general situation. Assume that the matrix C has s columns, and the norm of $C - Q Q^\top W C$ is zero or very small, then we approximate it by 0 and we then have the following fundamental identity:

$$U_{\ell+1} = [U_\ell \mid C] = Q \underbrace{\left[\Sigma \mid Q^\top W C \right]}_Y \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix}^\top. \quad (4.5)$$

In this paper we use the MATLAB built-in function `svd(Y, 'econ')` to find the SVD of Y since the matrix Y is always short-and-fat.

4.1 Update the left and right singular spaces

Since we assume that the matrix U is low rank, then it is reasonable to anticipate that most vectors of $\{u_{\ell+1}, u_{\ell+2}, \dots, u_n\}$ are linear dependent or almost linear dependent with the vectors in $Q \in \mathbb{R}^{m \times k}$. Without loss of generality, we assume that the next 2 vectors, $u_{\ell+1}$ and $u_{\ell+2}$, their residuals are less than the `tol` in Algorithm 4 when project them onto the space spanned by the columns of Q . In other words, we assume that

$$\|u_{\ell+1} - QQ^\top W u_{\ell+1}\|_W < \text{tol}, \quad \|u_{\ell+2} - QQ^\top W u_{\ell+2}\|_W < \text{tol}. \quad (4.6)$$

We then follow Brand's algorithm to find the SVD of $U_{\ell+2}$ incrementally:

- (1) Find the SVD of $[\Sigma \mid Q^\top W u_{\ell+1}]$, and we assume

$$\left[\Sigma \mid Q^\top W u_{\ell+1} \right] = Q_{(1)} \Sigma_{(1)} R_{(1)}^\top, \quad (4.7)$$

where $Q_{(1)} \in \mathbb{R}^{k \times k}$, $\Sigma_{(1)} \in \mathbb{R}^{k \times k}$, $R_{(1)} \in \mathbb{R}^{(k+1) \times k}$ and $Q_{(1)}^\top Q_{(1)} = Q_{(1)} Q_{(1)}^\top = I_k$.

- (2) Update the SVD of $U_{\ell+1}$ by

$$Q_{\ell+1} = QQ_{(1)}, \quad \Sigma_{\ell+1} = \Sigma_{(1)}, \quad R_{\ell+1} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_{(1)}. \quad (4.8)$$

By the above discussion, the update (4.8) is exactly the same with (2.3). Next, according to Algorithm 4, we should check the magnitude of the residue of $u_{\ell+2}$ when project onto the space spanned by the columns of $Q_{\ell+1}$. To do this, we introduce the following lemma.

Lemma 2. Let $Q \in \mathbb{R}^{k \times k}$ be an orthonormal matrix, then for any $x \in \mathbb{R}^m$, $Q \in \mathbb{R}^{m \times k}$ we have

$$QQ^\top x = (QQ)(QQ)^\top x.$$

Since $Q_{(1)}$ is an orthonormal matrix, then by Lemma 2 and (4.6) we have

$$\|u_{\ell+2} - Q_{\ell+1} Q_{\ell+1}^\top W u_{\ell+2}\|_W = \|u_{\ell+2} - QQ^\top W u_{\ell+2}\|_W < \text{tol}.$$

Next we follow the same arguments as in the above steps (1) and (2) to deal with $u_{\ell+2}$.

- (3) Find the SVD of $[\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2}]$, and we assume

$$\left[\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2} \right] = Q_{(2)} \Sigma_{(2)} R_{(2)}^\top, \quad (4.9)$$

where $Q_{(2)} \in \mathbb{R}^{k \times k}$, $\Sigma_{(2)} \in \mathbb{R}^{k \times k}$, $R_{(2)} \in \mathbb{R}^{(k+1) \times k}$ and $Q_{(2)}^\top Q_{(2)} = Q_{(2)} Q_{(2)}^\top = I_k$.

(4) By (4.8) we update the SVD of $U_{\ell+2}$ by

$$\begin{aligned} Q_{\ell+2} &= Q_{\ell+1}Q_{(2)} = QQ_{(1)}Q_{(2)}, \\ \Sigma_{\ell+2} &= \Sigma_{(2)}, \\ R_{\ell+2} &= \begin{bmatrix} R_{\ell+1} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)} = \begin{bmatrix} \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}. \end{aligned} \quad (4.10)$$

Algorithm 4 updates the left singular matrix $Q_{\ell+2}$ by computing $Q_{\ell+1} = QQ_{(1)}$ first and then compute $Q_{\ell+2} = Q_{\ell+1}Q_{(2)}$. The drawback of this update is its high computational cost. To avoid this, Brand proposed to compute $\tilde{Q} = Q_{(1)}Q_{(2)}$ first and then update the matrix Q by computing $Q\tilde{Q}$; see the details in Algorithm 6. For the right singular space, Algorithm 4 updates $R_{\ell+2}$ by computing $R_{\ell+1} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_{(1)}$ first and then $R_{\ell+2} = \begin{bmatrix} R_{\ell+1} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$. To save the CPU time, one can use Brand's algorithm to update the right singular subspace, but has to compute the pseudo-inverse of the sub-matrix of $R_{(1)}$ and $R_{(2)}$.

From Examples 1 and 2, we have seen that both algorithms have suffered in loss of the orthogonality of the left singular vector matrix.

Our new algorithm below can avoid the above issue without changing the complexity of Algorithm 6. To describe our new algorithm, we need the following fundamental identity.

$$\begin{bmatrix} \begin{bmatrix} A & 0 \\ 0 & 1 \end{bmatrix} B & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} A & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} B & 0 \\ 0 & 1 \end{bmatrix}, \quad A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{(n+1) \times n}. \quad (4.11)$$

By the fundamental identity (4.11) we can rewrite $R_{\ell+2}$ as

$$R_{\ell+2} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}.$$

By (4.10) we know that the SVD of $U_{\ell+2}$ is given by

$$U_{\ell+2} = QQ_{(1)}Q_{(2)}\Sigma_{(2)}R_{(2)}^\top \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix}^\top. \quad (4.12)$$

Motivated by (4.12), if we can compute $Q_{(1)}Q_{(2)}$ and $\begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$ directly, then the updates of the SVD of $U_{\ell+2}$ can possibly avoid many matrix-multiply-matrix operations.

Theorem 1. Let $\tilde{Q} = Q_{(1)}Q_{(2)}$, $\tilde{\Sigma} = \Sigma_{(2)}$ and $\tilde{R} = \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$, then $\tilde{Q}\tilde{\Sigma}\tilde{R}^\top$ is a thin SVD of

$$\begin{bmatrix} \Sigma & | & Q^\top W u_{\ell+1} & | & Q^\top W u_{\ell+2} \end{bmatrix}.$$

Proof. First, by using (4.7) we have

$$\begin{bmatrix} \Sigma & | & Q^\top W u_{\ell+1} & | & Q^\top W u_{\ell+2} \end{bmatrix} = \begin{bmatrix} Q_{(1)}\Sigma_{(1)}R_{(1)}^\top & | & Q^\top W u_{\ell+2} \end{bmatrix}.$$

Note that $Q_{(1)}^\top Q_{(1)} = Q_{(1)} Q_{(1)}^\top = I_k$ and by the fundamental identity (4.5) we have

$$\begin{aligned} \left[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2} \right] &= Q_{(1)} \left[\Sigma_{(1)} \mid Q_{(1)}^\top Q^\top W u_{\ell+2} \right] \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \\ &= Q_{(1)} \left[\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2} \right] \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top, \end{aligned}$$

where we used the first two identities in (4.8). Next we use (4.9) to obtain:

$$\left[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2} \right] = Q_{(1)} Q_{(2)} \Sigma_{(2)} R_{(2)}^\top \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top = \tilde{Q} \tilde{\Sigma} \tilde{R}^\top.$$

□

Theorem 1 implies that we can update the SVD of $U_{\ell+2}$ by computing a thin SVD of

$$\left[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2} \right] := \tilde{Q} \tilde{\Sigma} \tilde{R}^\top$$

first, and then update them by

$$Q_{\ell+2} = Q \tilde{Q}, \quad \Sigma_{\ell+2} = \tilde{\Sigma}, \quad R_{\ell+2} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \tilde{R}. \quad (4.13)$$

Now we extend the above discussions to a more general situation.

Theorem 2. Let $Q \Sigma R^\top$ be a rank k truncated core SVD of U_ℓ , $u_{\ell+j}$ be the $(\ell+j)$ -th column of U , $j = 1, 2, \dots, s$. If their residuals are less than `tol` in Algorithm 4 when project them onto the space spanned by the columns of Q . Let $\tilde{Q} \tilde{\Sigma} \tilde{R}^\top$ be the standard *thin* SVD of

$$\left[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2} \mid \dots \mid Q^\top W u_{\ell+s} \right] \in \mathbb{R}^{k \times (k+s)}. \quad (4.14)$$

Then we suggest the following updates

$$Q \leftarrow Q \tilde{Q}, \quad \Sigma \leftarrow \tilde{\Sigma}, \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix} \tilde{R}. \quad (4.15)$$

Moreover, the updates in (4.15) are exactly the same with the output of Algorithm 4.

Remark 2. In real computations, we do not compute $Q \tilde{Q}$, only need to store Q and $\tilde{Q}$ for future updates. Since we only need to compute $\tilde{Q}$ for r times, then we do not loss the orthogonality of their multiplications. Hence, our answer to Brand's question in [4] is that the reorthogonalizations to $\tilde{Q}$ are not needed. Furthermore, to update the right singular subspace, we do not compute the pseudo-inverse of a matrix. It is worthwhile mentioning that in practice we do not form the matrix $\begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix}$ by splitting $\tilde{R}$ into $\begin{bmatrix} \tilde{R}_1 \\ \tilde{R}_2 \end{bmatrix}$, and then R is updated by $\begin{bmatrix} R \tilde{R}_1 \\ \tilde{R}_2 \end{bmatrix}$. The total computational cost to update the matrix R is $\mathcal{O}(nr^3)$. Since $r \ll m, n$, then the cost can be neglected. However, it is worthwhile motioning that the technique in [4] can be applied to update the right singular space by computing r pseudo-inverse of a small matrix, and then the computational cost can be reduced to $\mathcal{O}(nr^2)$.

Once completed the SVD of $U_{\ell+s}$, we then update the SVD of $U_{\ell+s+1}$. To better describe the algorithm. We assume that $Q\Sigma R^\top$ is the core SVD of U_ℓ and $\widehat{Q}\widehat{\Sigma}\widehat{R}^\top$ is the core SVD of $U_{\ell+s}$, then

$$\widehat{Q} = Q\widetilde{Q}, \quad \widehat{R} = \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix} \widetilde{R}, \quad (4.16)$$

where $\widetilde{Q}$ and $\widetilde{R}$ are the left and right singular matrix of (4.14), respectively.

By our assumption, the residual of $u_{\ell+s+1}$ is larger than `tol` when project it onto the space spanned by the columns of Q . To keep the same accuracy, we should follow Brand's idea to compute the residual of $u_{\ell+s+1}$ that project it onto the space spanned by the columns of $\widehat{Q}$.

In other words, for those columns, we have to compute their residual twice. Hence the computational cost of those repetitions can be high. Next, we show that this cost can be avoided.

To this end, we follow the idea of Brand to construct the SVD of $[U_{\ell+s} \mid u_{\ell+s+1}]$ by

- (1) letting $e = u_{\ell+s+1} - \widehat{Q}\widehat{Q}^\top W u_{\ell+s+1}$ and let $p = \|e\|_W$ and $\tilde{e} = e/p$,
- (2) finding the full SVD of $\bar{Y} = \begin{bmatrix} \widehat{\Sigma} & \widehat{Q}^\top u_{\ell+s+1} \\ 0 & p \end{bmatrix}$, and let $\bar{Q}\bar{\Sigma}\bar{R}^\top$ be the SVD of $\bar{Y}$, then
- (3) updating the SVD of $[U_{\ell+s} \mid u_{\ell+s+1}]$ by

$$[U_{\ell+s} \mid u_{\ell+s+1}] = \left([\widehat{Q} \mid \tilde{e}] \bar{Q} \right) \bar{\Sigma} \left(\begin{bmatrix} \widehat{R} & 0 \\ 0 & 1 \end{bmatrix} \bar{R} \right)^\top. \quad (4.17)$$

Next, we discuss the above three steps. First, the computation in step (1) is *not* needed due to Lemma 2. Second, we do need to compute $\widehat{Q}^\top u_{\ell+s+1}$ in step (2). However, since $\widehat{Q}^\top u_{\ell+s+1} = \widetilde{Q}^\top Q^\top u_{\ell+s+1}$ and we have already computed $Q^\top u_{\ell+s+1}$, then the update of $\widehat{Q}^\top Q^\top u_{\ell+s+1}$ is almost nothing. For the step (3), by using (4.16) we have

$$\begin{aligned} [U_{\ell+s} \mid u_{\ell+s+1}] &= \left([Q\widetilde{Q} \mid \tilde{e}] \bar{Q} \right) \bar{\Sigma} \left(\begin{bmatrix} \widehat{R} & 0 \\ 0 & 1 \end{bmatrix} \bar{R} \right)^\top \\ &= \left([Q \mid \tilde{e}] \begin{bmatrix} \widetilde{Q} & 0 \\ 0 & 1 \end{bmatrix} \bar{Q} \right) \bar{\Sigma} \left(\begin{bmatrix} \widehat{R} & 0 \\ 0 & 1 \end{bmatrix} \bar{R} \right)^\top. \end{aligned} \quad (4.18)$$

This suggest to update the left singular matrix Q by adding the vector $\tilde{e}$, compute $\begin{bmatrix} \widetilde{Q} & 0 \\ 0 & 1 \end{bmatrix} \bar{Q}$ and then store it, the computational cost is $\mathcal{O}(r^2)$. For the right singular matrix, we directly compute it and the time complexity is $\mathcal{O}(nr^2)$. The step (3) will be executed r times, then the whole CPU time for this step is $\mathcal{O}(nr^3)$. We note that if we use the technique in [4], the computational cost can be reduced to $\mathcal{O}(nr^2)$ by computing r pseudo-inverse of a matrix.

4.2 Reorthogonalization

As we claimed in Section 2, small numerical errors in practice can cause a loss of orthogonality. A reorthogonalization to the matrix $([\widehat{Q} \mid \tilde{e}]\bar{Q})$ in [7, 13] is needed and the computational cost is high due to the two `for` loops in Algorithm 3.

Observing the identity (4.18), we note that the orthogonal matrices $\bar{Q}$ and $\widetilde{Q}$ are small and only r of them multiplied together, they can keep the orthogonality in a reasonable numerical precision.

Hence we do not need to perform reorthogonalizations to these small orthonormal matrices. This answers the question which was asked by Brand in [4].

Based on our numerical experiment in Example 2. The matrix $[Q \mid \tilde{e}]$ could loss its orthogonality very quickly. Therefore, we consider reorthogonalizations to the matrix $[Q \mid \tilde{e}]$. We note that this is necessary and was not included in Brand's original algorithm.

By the construction of $[Q \mid \tilde{e}]$, we can reorthogonalize it recursively: only apply the W -weighted Gram-Schmidt to the new adding vector $\tilde{e}$ since the matrix Q has already been reorthogonalized in previous steps if the weighted inner product between $\tilde{e}$ and the first column of Q is larger than some tolerance. Furthermore, if $\tilde{e} = 0$, then the last column will be truncated. In other words, we do not need to perform the reorthogonalization in the truncation part ($\|p\|_W < \text{tol}$); see Algorithm 7 for more details.

Algorithm 7 (Incremental SVD (III))

Input: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{\ell \times k}$, $u_{\ell+1} \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$, tol , V , Q_0 , q

```

1: Set  $d = Q^\top(Wu_{\ell+1})$ ;  $e = u_{\ell+1} - Qd$ ;  $p = (e^\top We)^{1/2}$ ;
2: if  $p < \text{tol}$  then
3:    $q = q + 1$ ;
4:   Set  $V\{q\} = Q_0^\top d$ ;
5: else
6:   if  $q > 0$  then
7:     Set  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
8:      $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ ;
9:     Set  $Q_0 = Q_0 Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $\mathcal{R}_1 = R_Y(1:k, 1:\text{end}-1)$ ,  $\mathcal{R}_2 = R_Y(k+1, 1:\text{end}-1)$ ,
10:     $R = \begin{bmatrix} R\mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}$ ;
11:    Set  $d = Q_Y^\top d$ 
12:   end if
13:   Set  $e = e/p$ ;
14:   if  $|e^\top WQ(:, 1)| > \text{tol}$  then
15:      $e = e - Q(Q^\top We)$ ;  $p_1 = (e^\top We)^{1/2}$ ;  $e = e/p_1$ ;
16:   end if
17:   Set  $Y = \begin{bmatrix} \Sigma & d \\ 0 & p \end{bmatrix}$ ;
18:    $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$ ;
19:   Set  $Q_0 = \begin{bmatrix} Q_0 & 0 \\ 0 & 1 \end{bmatrix} Q_Y$ ,  $Q = [Q \mid e]$ ,  $\Sigma = \Sigma_Y$ ,  $R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y$ .
20:   Set  $V = []$ ;  $q = 0$ ;
21: end if
22: return  $Q$ ,  $\Sigma$ ,  $R$ ,  $V$ ,  $Q_0$ ,  $q$ 

```

4.3 The complexity of Algorithm 7

We restrict ourselves the case $W = I$ to discuss the Algorithm 7. Recall that $U \in \mathbb{R}^{m \times n}$ with low rank r .

1. Line 1: this is exactly the same with Algorithm 5, the complexity is $\mathcal{O}(mnr)$.
2. Lines 3-4: the vector $d \in \mathbb{R}^k$ with $k \leq r$, hence this step is almost nothing for both CPU time and memory storage. Brand's method does not have this step.

3. Lines 7-11: the main cost is the update of R and the CPU time is less than (nr^3) . As we mentioned before, the cost can be reduced to $\mathcal{O}(nr^2)$. We leave it here since $r \ll m$.
4. Lines 14-16: the CPU time of the reorthogonalization is $\mathcal{O}(mr)$ each time and this part will be executed $\mathcal{O}(r)$ times, hence the whole complexity is $\mathcal{O}(mr^2)$. This part was not concluded in Algorithm 5 and it is necessary for some data sets.
5. Lines 17-20: the whole complexity is less than $\mathcal{O}(nr^3)$. Again, this can be reduced to $\mathcal{O}(nr^3)$ by using the technique in [4].

By our assumption, $r \ll m, n$, hence the CPU time of Algorithm 7 is $\mathcal{O}(mnr)$.

We note that the output of Algorithm 7, V , may be not empty. This implies that the output of Algorithm 7 is not the SVD of U . Hence we have to update the SVD for the vectors in V . We give the implementation of this step in Algorithm 8 and the complexity is $\mathcal{O}(mr^2)$. Finally, we complete the full implementation in Algorithm 9.

Algorithm 8 (Incremental SVD (III) final check)

Input: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{\ell \times k}$, V , Q_0 , q

```

1: if  $q > 0$  then
2:   Set  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
3:    $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ ;
4:   Set  $Q = Q(Q_0 Q_Y)$ ,  $\Sigma = \Sigma_Y$ ,  $\Sigma = \Sigma_Y$ ,  $\mathcal{R}_1 = R_Y(1:k, 1:\text{end}-1)$ ,
5:    $\mathcal{R}_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = \begin{bmatrix} R\mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}$ ;
6: else
7:   Set  $Q = Q Q_0$ ;
8: end if
9: return  $Q, \Sigma, R$ .
```

Algorithm 9 (Fully incremental SVD (III))

Input: $W \in \mathbb{R}^{m \times m}$, tol

```

1: Get  $u_1$ ;
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$ ; % Algorithm 1
3: Set  $V = []$ ;  $Q_0 = 1$ ;  $q = 0$ ;
4: for  $\ell = 2, \dots, n$  do
5:   Get  $u_\ell$ 
6:    $[q, V, Q_0, Q, \Sigma, R] = \text{UpdateISVD3}(q, V, Q_0, Q, \Sigma, R, u_\ell, W, \text{tol})$ ; % Algorithm 10
7: end for
8:  $[Q, \Sigma, R] = \text{UpdateISVD3check}(q, V, Q_0, Q, \Sigma, R)$ ; % Algorithm 12
9: return  $Q, \Sigma, R$ 
```

Example 3. In this example, we revisit Example 1 to test the efficiency and accuracy of Algorithm 9. We show the orthogonality of the left singular vector matrix in Figure 4, where

$$\mathcal{E}_W(Q) = \|I - Q^\top W Q\|, \quad \mathcal{E}_I(\tilde{Q}) = \|I - \tilde{Q}^\top \tilde{Q}\|.$$

We see that our Algorithm 9 keeps a good orthogonality.

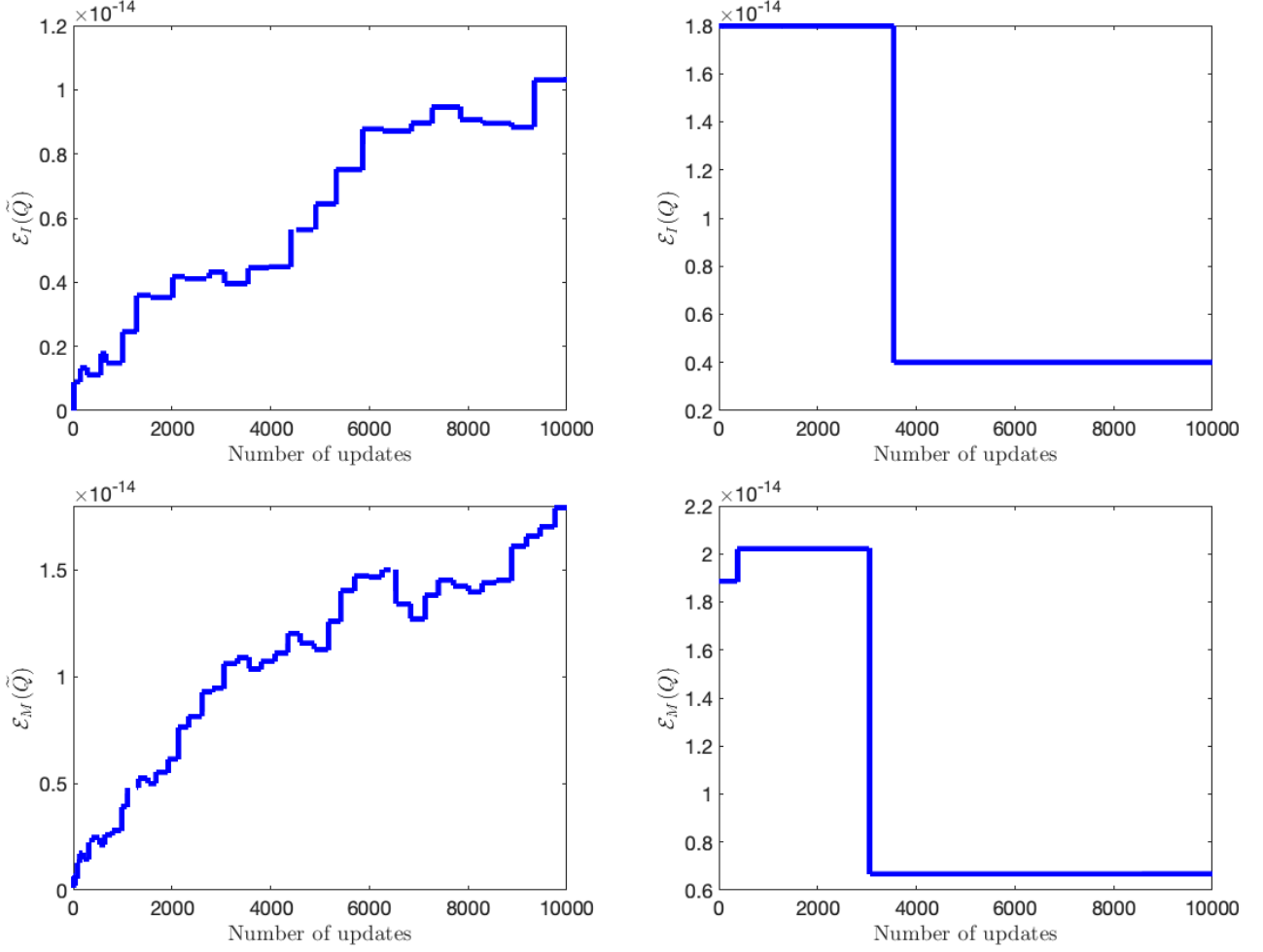


Figure 4: Example 3: Top: the orthogonality of $\tilde{Q}$ and Q with $W = I$. Bottom: the orthogonality of $\tilde{Q}$ and Q with $W = M$.

Next, we show the CPU time (seconds) of the main part of the Algorithm 7 in Table 2. The whole simulations take about 34 and 118 seconds for $W = I$ and $W = M$, respectively. Comparing with Table 1, the CPU times is reduced greatly, especially with a weighted norm setting ($W \neq I$). Furthermore, the projection part (line 1 in Algorithm 7) takes about 99% of the whole simulation time, this implies there is litter space to improve the efficiency of Algorithm 9.

Line(s) in Algorithm 7	1	3-4	7-11	14-16	17-20
$W = I$	33.457	0.2416	0.0327	0.1251	0.2882
$W = M$	116.41	0.1870	0.0650	0.6526	0.7120

Table 2: Example 3: The CPU time (seconds) for two different weighted norms by using Algorithm 9.

5 Another truncation

For many data sets, they may have a large number of nonzero singular values but most of them are very small. Without truncating those small singular values, the incremental SVD algorithm maybe keep computing them and hence the computational cost is huge. Following [7, Algorithm 4], we set another truncation if the last few singular values are less than a tolerance.

Next, we show that it is only necessary to check the last singular value. Let $Q\Sigma R^\top$ be a core SVD of U_ℓ , $e = u_{\ell+1} - QQ^\top W u_{\ell+1}$ and $p = \|e\|_W > \text{tol}$, then

$$U_{\ell+1} = [Q \mid \tilde{e}] \underbrace{\begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_Y \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top = [Q \mid \tilde{e}] Q_Y \Sigma_Y R_Y^\top \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top.$$

Next, we estimate the singular values of Y .

Lemma 3. Assume that $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_k)$ with $\sigma_1 \geq \dots \geq \sigma_k$, and $\Sigma_Y = \text{diag}(\mu_1, \dots, \mu_{k+1})$ with $\mu_1 \geq \dots \geq \mu_{k+1}$. Then we have

$$\mu_{k+1} \leq p, \quad (5.1a)$$

$$\mu_{k+1} \leq \sigma_k \leq \mu_k \leq \sigma_{k-1} \leq \dots \leq \sigma_1 \leq \mu_1. \quad (5.1b)$$

Proof. First, we consider $Y^\top Y$,

$$Y^\top Y = \begin{bmatrix} \Sigma^2 & \Sigma Q^\top W u_{\ell+1} \\ u_{\ell+1}^\top W Q \Sigma & p^2 + u_{\ell+1}^\top W Q Q^\top W u_{\ell+1} \end{bmatrix}.$$

By using the Cauchy interlace theorem [9, Theorem 1] we have

$$\mu_{k+1} \leq \sigma_k \leq \mu_k \leq \sigma_{k-1} \leq \dots \leq \sigma_1 \leq \mu_1.$$

Next, we take $e = [0, 0, \dots, 1]$ to obtain

$$p^2 = e Y Y^\top e \geq \min_{\|x\|=1} x Y Y^\top x = \lambda_{\min}(Y Y^\top) = \mu_{k+1}^2.$$

□

Inequality (5.1a) implies that the last singular value of Y can possibly be very small, no matter how large of p . That is to say the tolerance we set for p can not avoid the algorithm keeping compute very small singular values. Hence, another truncation is needed if the data has a large number of very small singular values. Inequality (5.1b) guarantees us that *only* the last singular value of Y can possibly less than the tolerance. Hence, we only need to check the last one.

(i) If $\Sigma_Y(k+1, k+1) \geq \text{tol}$, then

$$Q \leftarrow [Q \mid \tilde{e}] Q_Y \quad \Sigma \leftarrow \Sigma_Y \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y. \quad (5.2)$$

(ii) If $\Sigma_Y(k+1, k+1) < \text{tol}$, then

$$Q \leftarrow [Q \mid \tilde{e}] Q_Y(:, 1:k) \quad \Sigma \leftarrow \Sigma_Y(1:k, 1:k) \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(:, 1:k). \quad (5.3)$$

As we discussed in Section 4, we can avoid time complexity $\mathcal{O}(mr^2)$ in (5.2) by only updating the small matrix Q_Y and storing Q_Y and $[Q \mid \tilde{e}]$ separately. Unfortunately, to truncate the very small singular values, it seems very difficult that we can find two orthonormal matrices $\hat{Q} \in \mathbb{R}^{m \times k}$ and $\tilde{Q} \in \mathbb{R}^{k \times k}$ in $\mathcal{O}(r^2)$ time such that

$$[Q \mid \tilde{e}] Q_Y(:, 1:k) = \hat{Q} \tilde{Q}.$$

Hence, we directly use (5.2) and (5.3) to update the matrix Q and R . Assume the data has d nonzero singular values, then this new truncation needs $\mathcal{O}((m+n)r^2d)$ time, here r is the number of singular values which are larger than the tolerance.

Algorithm 10 (Incremental SVD (IV))

Input: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{\ell \times k}$, $u_{\ell+1} \in \mathbb{R}^m$, $W \in \mathbb{R}^{m \times m}$, **tol**, V , Q_0 , q

```

1: Set  $d = Q^\top(Wu_{\ell+1})$ ;  $e = u_{\ell+1} - Qd$ ;  $p = (e^\top We)^{1/2}$ ;
2: if  $p < \text{tol}$  then
3:    $q = q + 1$ ;
4:   Set  $V\{q\} = d$ ;
5: else
6:   if  $q > 0$  then
7:     Set  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
8:      $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ ;
9:     Set  $Q_0 = Q_0 Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $\mathcal{R}_1 = R_Y(1:k, 1:\text{end}-1)$ ,
10:     $\mathcal{R}_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = \begin{bmatrix} R\mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}$ ;
11:    Set  $d = Q_Y^\top d$ 
12:   end if
13:   Set  $e = e/p$ ;
14:   if  $|e^\top WQ(:, 1)| > \text{tol}$  then
15:      $e = e - Q(Q^\top We)$ ;  $p_1 = (e^\top We)^{1/2}$ ;  $e = e/p_1$ ;
16:   end if
17:   Set  $Y = \begin{bmatrix} \Sigma & d \\ 0 & p \end{bmatrix}$ ;
18:    $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$ ;
19:   if  $\Sigma(k+1, k+1) > \text{tol}$  then
20:     Set  $Q_0 = \begin{bmatrix} Q_0 & 0 \\ 0 & 1 \end{bmatrix} Q_Y$ ,  $Q = [Q \mid e]Q_0$ ,  $\Sigma = \Sigma_Y$ ,  $\mathcal{R}_1 = R_Y(1:k, 1:\text{end}-1)$ ,
21:     $\mathcal{R}_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = \begin{bmatrix} R\mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}$ ,  $Q_0 = I_{k+1}$ ;
22:   else
23:     Set  $Q_0 = \begin{bmatrix} Q_0 & 0 \\ 0 & 1 \end{bmatrix} Q_Y$ ,  $Q = [Q \mid e]Q_0(:, 1:k)$ ,
24:     $\Sigma = \Sigma_Y(1:k, 1:k)$ ,  $R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(:, 1:k)$ ,  $Q_0 = I_k$ 
25:   end if
26:    $V = []$ ;  $q = 1$ ;
27: end if
28: return  $Q, \Sigma, R, V, Q_0, q$ 

```

Next, we complete the full implementation in Algorithms 11 and 12.

Algorithm 11 (Incremental SVD (IV) final check)

Input: $Q \in \mathbb{R}^{m \times k}$, $\Sigma \in \mathbb{R}^{k \times k}$, $R \in \mathbb{R}^{\ell \times k}$, V , q

```

1: if  $q > 0$  then
2:   Set  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
3:    $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ ;
4:   Set  $Q = QQ_Y$ ,  $\Sigma = \Sigma_Y$ ,  $\mathcal{R}_1 = R_Y(1:k, 1:\text{end}-1)$ ,
5:    $\mathcal{R}_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = \begin{bmatrix} R\mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}$ ;
6: end if
7: return  $Q, \Sigma, R$ .
```

Algorithm 12 (Fully incremental SVD (IV))

Input: $W \in \mathbb{R}^{m \times m}$, tol

```

1: Get  $u_1$ ;
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$ ; % Algorithm 1
3: Set  $V = []$ ;  $Q_0 = 1$ ;  $q = 1$ ;
4: for  $\ell = 2, \dots, n$  do
5:   Get  $u_\ell$ 
6:    $[q, V, Q_0, Q, \Sigma, R] = \text{UpdateISVD4}(q, V, Q_0, Q, \Sigma, R, u_\ell, W, \text{tol})$ ; % Algorithm 10
7: end for
8:  $[Q, \Sigma, R] = \text{UpdateISVD4check}(q, V, Q_0, Q, \Sigma, R)$ ; % Algorithm 11
9: return  $Q, \Sigma, R$ 
```

6 Conclusion

In this paper, we answered the open question which was asked by Brand in [4]. We prove that the output of our new algorithm is the same with Brand's original algorithm. Numerical experiments showed that our new algorithm not only saves the CPU, but also keeps a very good orthogonality for the left singular vector matrix. In the future, we will compare our new algorithm with many other incremental SVD algorithms in [1]. Besides this, we will apply this new algorithm for many different problems, such as model order reduction and fractional PDEs.

Acknowledgment

The author would like to thank Dr. Lihan Wang for the proof of Lemma 3.

References

- [1] C. BACH, D. CEGLIA, L. SONG, AND F. DUDDECK, *Randomized low-rank approximation methods for projection-based model order reduction of large nonlinear dynamical problems*, Internat. J. Numer. Methods Engrg., 118 (2019), pp. 209–241, <https://doi.org/10.1002/nme.6009>.
- [2] M. W. BERRY, *Large-scale sparse singular value computations*, The International Journal of Supercomputing Applications, 6 (1992), pp. 13–49.

- [3] M. BRAND, *Incremental singular value decomposition of uncertain data with missing values*, in European Conference on Computer Vision, Springer, 2002, pp. 707–720.
- [4] M. BRAND, *Fast low-rank modifications of the thin singular value decomposition*, Linear Algebra Appl., 415 (2006), pp. 20–30, <https://doi.org/10.1016/j.laa.2005.07.021>.
- [5] H. FAREED AND J. R. SINGLER, *A note on incremental POD algorithms for continuous time data*, Appl. Numer. Math., 144 (2019), pp. 223–233, <https://doi.org/10.1016/j.apnum.2019.04.020>.
- [6] H. FAREED AND J. R. SINGLER, *Error analysis of an incremental proper orthogonal decomposition algorithm for PDE simulation data*, J. Comput. Appl. Math., 368 (2020), pp. 112525, 14, <https://doi.org/10.1016/j.cam.2019.112525>.
- [7] H. FAREED, J. R. SINGLER, Y. ZHANG, AND J. SHEN, *Incremental proper orthogonal decomposition for PDE simulation data*, Comput. Math. Appl., 75 (2018), pp. 1942–1960, <https://doi.org/10.1016/j.camwa.2017.09.012>.
- [8] M. GHASHAMI, E. LIBERTY, J. M. PHILLIPS, AND D. P. WOODRUFF, *Frequent directions: simple and deterministic matrix sketching*, SIAM J. Comput., 45 (2016), pp. 1762–1792, <https://doi.org/10.1137/15M1009718>.
- [9] S.-G. HWANG, *Cauchy’s interlace theorem for eigenvalues of Hermitian matrices*, Amer. Math. Monthly, 111 (2004), pp. 157–159, <https://doi.org/10.2307/4145217>.
- [10] M. A. IWEN AND B. W. ONG, *A distributed and incremental SVD algorithm for agglomerative data analysis on large networks*, SIAM J. Matrix Anal. Appl., 37 (2016), pp. 1699–1718, <https://doi.org/10.1137/16M1058467>.
- [11] L. LIN AND Y. TONG, *Low-rank representation of tensor network operators with long-range pairwise interactions*, SIAM J. Sci. Comput., 43 (2021), pp. A164–A192, <https://doi.org/10.1137/19M1287067>.
- [12] N. MASTRONARDI, E. E. TYRTYSHNIKOV, AND P. VAN DOOREN, *A fast algorithm for updating and downsizing the dominant kernel principal components*, SIAM J. Matrix Anal. Appl., 31 (2010), pp. 2376–2399, <https://doi.org/10.1137/090774422>.
- [13] G. M. OXBERRY, T. KOSTOVA-VASSILEVSKA, W. ARRIGHI, AND K. CHAND, *Limited-memory adaptive snapshot selection for proper orthogonal decomposition*, Internat. J. Numer. Methods Engrg., 109 (2017), pp. 198–217, <https://doi.org/10.1002/nme.5283>.

毕业论文（设计）文献综述和开题报告考核

对文献综述、外文翻译和开题报告评语及成绩评定

成绩比例	文献综述占 (10%)	开题报告占 (15%)	外文翻译占 (5%)
分值			

开题报告答辩小组负责人（签名）_____

2025 年 月 日